

University of Tripoli

Faculty of Engineering

Department of Computer Engineering

A graduation project submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Engineering

Phishing URL Classification Using Generative Adversarial Networks

By

Ali AbdulMajid Alganbour

Student ID: 2190209816

Supervised by

Dr. Yusra Maatoug

Spring 2026

Phishing URL Classification Using Generative Adversarial Networks

By

Ali AbdulMajid Alganbour

A graduation project approved by the Department of Computer Engineering

in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Engineering.

Supervisor: Dr. Yusra Maatoug Signature: _____

Examiner: _____ Signature: _____

Examiner: _____ Signature: _____

Spring 2026



Department of Computer Engineering
Faculty of Engineering
University of Tripoli

Intellectual Property Rights Identification Form for Projects and Scientific Research

This form must be read and signed by students working on graduation projects, master's theses or any other research activities conducted at University of Tripoli / Faculty of Engineering / Department of Computer Engineering.

Intellectual property rights for projects and research activities and their results (such as graduation projects, master's theses, patents and any marketable research product) belong to the University of Tripoli / Department of Computer Engineering. These rights are subject to the laws, regulations and instructions of the University relating to intellectual property and patents.

I agree (Student's Name): _____

Student's ID: _____

As a condition of my participation in the research project entitled:

_____

_____

_____

All intellectual property rights of the above-mentioned project or scientific research shall be attributable to the University of Tripoli / Department of Computer Engineering. This requires me to inform the competent authority of the University of any invention or discovery that may result from such research and to be fully confidential therein and to work through the University to obtain the patent that may result from such research. I am also committed to placing the name of Tripoli University / Department of Computer Engineering and the names of all researchers involved in the research on any scientific bulletin for full research or its results, including publication of graduation projects, master's theses, doctorates, publication in journals, scientific conferences in general and posting on websites. I must adhere to the principles of copyright approved by the University of Tripoli / Department of Computer Engineering.

Signature: _____

Date: _____



Department of Computer Engineering
Faculty of Engineering
University of Tripoli

Plagiarism Declaration

I (Student's Name): _____

Student's ID: _____

hereby declare that I am the sole author of the graduation project entitled:

_____

_____

_____

and that neither any part of the thesis nor the whole of the thesis has been submitted to any University or Institution for obtaining any degree / diploma / academic award.

This project was written by me and in my own words, except for quotations from published and unpublished sources which are clearly indicated and acknowledged as such. I am conscious that the incorporation of material from other works or a paraphrase of such material without acknowledgement will be treated as plagiarism, subject to the custom and usage of the subject, according to the University Regulations on Conduct of Examinations.

I shall be solely responsible for any dispute or plagiarism issue arising out of the graduation project.

Signature: _____

Date: _____

Abstract

Phishing websites use misleading URLs to imitate trusted services and obtain sensitive information from users. Detection models trained only on historical URLs may perform well on familiar data but lose accuracy when URL structures, sources, or attack patterns change. This project investigates whether phishing URLs generated by Generative Adversarial Networks (GANs) can improve the generalisation of URL-based classifiers. Traditional and deep-learning classifiers were trained as real-only baselines, including Random Forest, Support Vector Machine, Convolutional Neural Network, Recurrent Neural Network, and Long Short-Term Memory models. Several text-generation approaches were then studied, and generated URLs were checked for validity, uniqueness, diversity, overlap with real data, and their ability to challenge real-only classifiers. Controlled augmentation experiments added fixed percentages of synthetic phishing URLs while keeping the real training rows and evaluation sets unchanged. A matched comparison also trained SeqGAN and MS-HardGAN-C on the exact same 84,280 phishing rows; MS-HardGAN-C retained the better downstream F1 and false-negative balance. The final system combines character-level CNN learning with 35 lexical URL features, multi-source real training, score calibration, and a three-model ensemble. On a final test of 6,000 unseen URLs containing 5% phishing, the selected 10% GAN-augmented model achieved 98.23% accuracy, 84.15% precision, 79.67% recall, and 81.85% F1-score. Its false-positive rate was 0.79% and its false-negative rate was 20.33%. The model was also exported to a browser extension that performs local URL analysis and presents warning levels without uploading visited URLs. A later browser-policy audit showed that blanket trust for shared hosting could hide phishing URLs. Replacing it with a non-blocking shared-content warning reduced FNR from 21.33% to 13.33% on the audited snapshot without changing its measured FPR of 0.69%. Overall, carefully filtered GAN augmentation improved detection under some test conditions, but generated data cannot replace diverse real data or remove all effects of distribution shift.

Acknowledgement

First, I thank Allah for giving me the strength and patience to complete this project.

I would like to express my sincere appreciation to my supervisor, Dr. Yusra Maatoug, for her guidance, feedback, and support throughout the project. I also thank the staff of the Department of Computer Engineering for the knowledge and assistance provided during my studies.

I am deeply grateful to my family and friends for their encouragement and support. Finally, I acknowledge the researchers and open-source communities whose publications, datasets, and software tools supported the implementation and evaluation of this work.

Contents

Abstract i

Acknowledgement ii

List of Abbreviations xii

1 Introduction 1

1.1 Project Background . . . 1
1.2 Problem Statement . . . 2
1.3 Aim . . . 2
1.4 Objectives . . . 2
1.5 Research Questions . . . 3
1.6 Project Approach . . . 4
1.7 Project Scope . . . 4
1.7.1 Included in the Project . . . 4
1.7.2 Outside the Project Scope . . . 5
1.8 Main Contributions . . . 5
1.9 Report Structure . . . 6

2 Technical Background 7

2.1 Phishing and URL-Based Detection . . . 7
2.2 Structure of a URL . . . 8
2.3 Representing URLs for Machine Learning . . . 8
2.3.1 Lexical Features . . . 8
2.3.2 Character-Level Representation . . . 9
2.3.3 Hybrid Representation . . . 10
2.4 Classifier Families . . . 10
2.4.1 Random Forest . . . 10
2.4.2 Support Vector Machine . . . 10
2.4.3 Convolutional Neural Network . . . 10
2.4.4 RNN and LSTM . . . 11
2.4.5 Hybrid CNN Classifier . . . 11

2.5 Generative Adversarial Networks . . . 11
2.5.1 Basic Adversarial Training . . . 11
2.5.2 Why URL Text Is Difficult to Generate . . . 12
2.5.3 GAN Designs Relevant to the Project . . . 13
2.6 GAN Augmentation for Defensive Training . . . 13
2.7 Evaluating Generated URL Quality . . . 14
2.8 Distribution Shift and Data Leakage . . . 14
2.8.1 Distribution Shift . . . 14
2.8.2 Data Leakage . . . 15
2.9 Evaluation Metrics . . . 15
2.10 Thresholds, Calibration, and Browser Decisions . . . 16
2.11 Chapter Summary . . . 16

3 Related Work **17**
3.1 Review Scope and Comparison Method . . . 17
3.2 URL Features and Traditional Classifiers . . . 18
3.3 Character-Level Deep Learning . . . 19
3.4 GAN-Based Phishing Research . . . 19
3.4.1 PWDGAN: Black-Box Evasion . . . 19
3.4.2 Conditional GAN with Joint Detection . . . 20
3.4.3 PDGAN: LSTM Generator and CNN Discriminator . . . 20
3.4.4 WGAN-GP Augmentation . . . 21
3.4.5 Comparison of Basic GAN, DCGAN, WGAN, and SeqGAN . . . 21
3.4.6 Generative Round Network . . . 22
3.5 Comparison of the Reviewed Studies . . . 22
3.6 Research Gap . . . 24
3.7 Chapter Summary . . . 25

4 Methodology and System Design **26**
4.1 Research Design . . . 26
4.2 Development Through Three Versions . . . 27
4.3 Data Sources and Dataset Roles . . . 28
4.3.1 Source separation . . . 28
4.3.2 Label convention . . . 30
4.3.3 Illustrative real URL records . . . 30
4.4 URL Preparation and Leakage Control . . . 30
4.4.1 Normalization . . . 30
4.4.2 Duplicate and conflict removal . . . 31
4.4.3 Hostname-separated splitting . . . 31
4.5 Input Representations . . . 31

4.5.1 Character representation . . . 31
4.5.2 Lexical representation . . . 32
4.6 Classifier Design . . . 32
4.6.1 Fairness of classifier comparisons . . . 33
4.6.2 Hybrid multi-source model . . . 33
4.7 GAN Design and Training . . . 34
4.7.1 Training corpus . . . 34
4.7.2 Implemented generators . . . 34
4.7.3 Classifier-challenging objective . . . 35
4.7.4 Epoch interpretation . . . 36
4.8 Synthetic URL Quality Control . . . 36
4.8.1 Illustrative generated samples . . . 36
4.9 Augmentation Conditions . . . 37
4.10 Training and Model Selection . . . 38
4.11 Evaluation Protocol . . . 39
4.11.1 Classifier metrics . . . 39
4.11.2 Evaluation levels . . . 39
4.11.3 DNS secondary experiment . . . 39
4.11.4 Browser user-evaluation measures . . . 40
4.12 Browser System Design and Privacy . . . 40
4.13 Reproducibility Controls . . . 41
4.14 Chapter Summary . . . 41

5 System Implementation 42
5.1 Implementation Overview . . . 42
5.2 Project Structure . . . 42
5.3 Data Preparation . . . 43
5.4 Feature Extraction . . . 44
5.5 Classifier Implementation . . . 44
5.6 GAN Implementation and Augmentation . . . 45
5.7 Training and Evaluation Process . . . 46
5.8 Browser Extension Implementation . . . 46
5.9 Local Feedback and Privacy . . . 47
5.10 Verification and Reproducibility . . . 47
5.11 Chapter Summary . . . 48

6 Experiments and Results 49
6.1 Introduction . . . 49
6.2 Development Results from Versions 1 and 2 . . . 49
6.3 Version 3 Real-Only Baselines . . . 50

6.4 Quality of Generated Phishing URLs . . . 51
6.4.1 Initial generator comparison . . . 51
6.4.2 MS-HardGAN-C quality . . . 51
6.5 Controlled Augmentation Results . . . 53
6.5.1 SeqGAN augmentation of the baselines . . . 53
6.5.2 MS-HardGAN-C augmentation . . . 54
6.5.3 Matched SeqGAN and MS-HardGAN-C comparison . . . 55
6.5.4 Final hybrid ratio comparison . . . 57
6.6 External Evaluation . . . 57
6.6.1 Why prevalence matters . . . 57
6.6.2 External comparison of classifier families . . . 57
6.6.3 Primary 30K low-prevalence confirmation . . . 59
6.6.4 Independent PhishTank and later frozen snapshots . . . 60
6.6.5 Long-URL stress test . . . 60
6.7 Threshold Analysis . . . 61
6.8 Discussion of the Research Questions . . . 62
6.8.1 Can a URL-only classifier detect phishing effectively? . . . 62
6.8.2 Does GAN augmentation improve detection? . . . 62
6.8.3 Are the generated URLs useful beyond final accuracy? . . . 63
6.8.4 What remains difficult? . . . 63
6.9 Chapter Summary . . . 63

7 Browser Extension and User Testing 64
7.1 Introduction . . . 64
7.2 Local Detection Process . . . 64
7.3 Explainability and User Control . . . 65
7.3.1 Interface examples . . . 65
7.4 Reviewed Services and User-Controlled Content . . . 69
7.5 Reliability and Privacy Controls . . . 69
7.6 User Evaluation . . . 69
7.6.1 Method and browsing context . . . 69
7.6.2 Recorded outcomes . . . 70
7.6.3 Reported error categories . . . 71
7.7 Technical Verification . . . 72
7.8 Chapter Summary . . . 72

8 Conclusion 74
8.1 Project Overview . . . 74
8.2 Answers to the Research Questions . . . 74
8.2.1 RQ1: Performance of real-only classifiers . . . 74

8.2.2 RQ2: Quality of generated phishing URLs . . . 75
8.2.3 RQ3: Effect of GAN augmentation . . . 75
8.2.4 RQ4: Generalisation to different and imbalanced sources . . . 76
8.2.5 RQ5: Browser implementation . . . 76
8.3 Achievement of the Project Objectives . . . 77
8.4 Main Findings . . . 77
8.5 Limitations . . . 78
8.6 Final Conclusion . . . 78

9 Future Work **80**
9.1 Introduction . . . 80
9.2 Newer and Broader URL Data . . . 80
9.3 Reducing False Warnings and Missed Phishing . . . 80
9.4 Additional Detection Evidence . . . 81
9.5 Further GAN Research . . . 81
9.6 Browser Extension Improvements . . . 82
9.7 Larger User Evaluation . . . 82
9.8 Long-Term Maintenance . . . 82
9.9 Chapter Summary . . . 83

References **84**

List of Figures

1.1 High-level workflow of the project. 4

2.1 Main components of a URL. 8
2.2 Basic adversarial training cycle adapted to phishing URL generation. . 12

4.1 Overall experimental workflow. 27
4.2 High-level architecture of the hybrid URL classifier. 34

5.1 Parallel real-only and GAN-augmented training, evaluation, and deployment workflow. 46

6.1 Comparison of the Version 3 real-only baseline classifiers. 51
6.2 Generated-URL quality measures for SeqGAN, URL-GAN, and RelGAN. 52
6.3 Quality profile of the accepted MS-HardGAN-C URL pool. 53
6.4 Internal multi-metric results for the SeqGAN augmentation ratios. . . . 54
6.5 Internal real-test results under MS-HardGAN-C augmentation. 54
6.6 Recall on the frozen MS-HardGAN-C synthetic challenge set. 55
6.7 Real-only HYB-MS and the two matched-input 10% generator conditions. 56
6.8 External performance of the equally calibrated classifier families. 58
6.9 All classifiers on the frozen 30K low-prevalence external set. 59
6.10 F1, FPR, and FNR across the low-prevalence external evaluations. Grey dashed lines with square markers represent the real-only model; blue solid lines with circular markers represent the 10% GAN-augmented model. 60
6.11 Recall, FPR, and FNR trade-off across the registered score policies. . . 62

7.1 Extension popup examples for a normal website and a local IP address. 66
7.2 Full warning page shown when Normal mode pauses a very-high-risk URL. 67
7.3 Non-blocking notice for a very-high-risk result in Warnings-only mode. 68

7.4 Retained user-reported and participant-confirmed outcomes by export and website category. These counts are feedback observations, not independently verified test labels. 72

List of Tables

1.1 Project objectives and how each objective is evaluated. 3

2.1 Groups of lexical URL features used in phishing detection. 9

3.1 Comparison of closely related URL-detection and GAN studies. 23

4.1 Purpose of the three project versions. 28
4.2 Frozen original Version 3 splits. 28
4.3 Principal datasets and their permitted roles. 29
4.4 Illustrative legitimate and phishing URL records used by the project. . 30
4.5 Classifier configurations used in Version 3. 33
4.6 Implemented Version 3 GAN configurations. 35
4.7 Illustrative defanged MS-HardGAN-C samples from the frozen 10% pool. 37
4.8 Registered augmentation counts for the 363,692-row multi-source real pool. 37

5.1 Main technologies used in the implemented system. 43
5.2 Implemented classifiers and their input representations. 44

6.1 Summary of the first two experimental versions. 50
6.2 Version 3 real-only baseline results on the internal test set (mean of three seeds, %). 50
6.3 Quality comparison of the initial Version 3 generators. 51
6.4 MS-HardGAN-C generation and filtering results. 52
6.5 Internal F1-score under SeqGAN augmentation (mean of three seeds). . 53
6.6 Matched-input generator comparison using the same HYB-MS procedure (%). 56
6.7 Calibrated HYB-MS results on the frozen balanced 10K external set (%). 57
6.8 Equal-calibration external comparison on the balanced 10K set (%). . . 58
6.9 All classifier results on the 30K external set with 5% phishing prevalence. 59
6.10 External comparison of real-only and 10% GAN-augmented HYB-MS (%). 60

6.11 Results on the long-URL stress set (%). 61
6.12 Single-stage threshold policies on the registered threshold evaluation set. 61

7.1 Browser decision tiers and their user-facing behaviour. 65
7.2 Participant-level browser user-evaluation results. 70
7.3 Summary of current-format browser user-evaluation observations. . . . 71
7.4 Categories of retained user-reported or participant-confirmed outcomes. 71

8.1 Final status of the objectives defined in Chapter 1. 77

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
CNN	Convolutional Neural Network
DNS	Domain Name System
FNR	False-Negative Rate
FPR	False-Positive Rate
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
HYB-MS	Hybrid Multi-Source Model
LSTM	Long Short-Term Memory
ML	Machine Learning
ONNX	Open Neural Network Exchange
PR-AUC	Area Under the Precision–Recall Curve
RF	Random Forest
RNN	Recurrent Neural Network
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SVM	Support Vector Machine
URL	Uniform Resource Locator

Chapter 1

Introduction

1.1 Project Background

Phishing is a social-engineering attack in which an attacker imitates a trusted service to persuade a user to reveal information such as a password, bank-card number, or account code. A phishing attempt may be delivered through email, text messages, social media, advertisements, or a compromised website. In many cases, the user is directed to a web address that looks believable but leads to a fraudulent page.

The URL is useful for early detection because it can be examined before a user submits information or downloads content. URL-based detection is also suitable for a browser extension because it does not require the complete webpage to be downloaded and analysed. Traditional blacklist systems remain useful for known threats, but a newly created phishing URL may not yet appear in a blacklist. Machine-learning models can instead learn structural and character patterns that separate legitimate and phishing URLs [1, 2].

The main difficulty is that phishing URLs change over time. Attackers can alter domain names, subdomains, paths, query strings, spelling, special characters, and hosting services. At the same time, legitimate browsing also contains long and complex URLs. A detector that treats every unfamiliar or complicated URL as phishing may achieve high phishing recall while producing too many false warnings for normal users.

Generative Adversarial Networks offer a possible way to create additional training examples. A GAN contains a generator that produces synthetic samples and a discriminator that learns to distinguish generated samples from real ones. GAN-based phishing research has considered both generation and detection, including conditional generation, black-box detector evasion, and comparisons of several GAN families [2–4]. This project studies whether generated phishing URLs can be used defensively as controlled training augmentation.

1.2 Problem Statement

Supervised phishing detectors normally learn from labelled historical data. Their reported accuracy can be high when the test data closely resembles the training data. However, performance may fall when URLs are collected from a different source, when legitimate browsing contains structures that were not represented in training, or when phishing campaigns adopt new combinations of known patterns.

Simply adding more generated phishing URLs does not guarantee improvement. Synthetic samples may be invalid, repetitive, unrealistic, too similar to the training data, or limited to a small number of templates. They may also change the class balance in a way that increases missed phishing URLs or false warnings. The effect of augmentation must therefore be tested under controlled conditions where the real training data, validation data, test data, classifier settings, and evaluation metrics remain fixed.

The project addresses the following problem:

> *How can realistic and varied GAN-generated phishing URLs be used to improve URL classification without hiding increases in false positives, false negatives, or failures on URLs collected from different sources?*

The problem has both a research part and an implementation part. The research part compares real-only training with GAN-augmented training. The implementation part places the selected model in a browser extension and studies how model scores should be shown to users without presenting every uncertain URL as confirmed phishing.

1.3 Aim

The aim of this project is to design, implement, and evaluate a phishing URL detection system that uses GAN-generated phishing URLs as controlled data augmentation, while maintaining a practical balance between phishing detection and false warnings on legitimate URLs.

1.4 Objectives

The objectives originate from the approved project proposal and are organised into the measurable tasks shown in Table 1.1.

Table 1.1: Project objectives and how each objective is evaluated.

No.	Objective	Evaluation evidence
O1	Prepare labelled legitimate and phishing URL data with duplicate and overlap checks.	Dataset counts, class balance, URL-length analysis, and split checks.
O2	Train traditional and deep-learning real-only classifiers.	RF, SVM, CNN, RNN, LSTM, and hybrid model metrics.
O3	Train GAN models using phishing URLs from the training data only.	Generator and discriminator training records.
O4	Measure generated URL quality and diversity.	Validity, uniqueness, overlap, length, structure, and classifier-challenge tests.
O5	Compare real-only and GAN-augmented training fairly.	Fixed augmentation percentages, identical real rows, fixed test sets, and repeated seeds.
O6	Test generalisation on URLs not used for training.	Internal, external, low-phishing, long-URL, and generator-challenge evaluations.
O7	Select and implement a model for real-time use.	ONNX export, browser inference checks, warning policy, and user testing.

1.5 Research Questions

The work is guided by the following research questions:

RQ1: How well do traditional, neural, and hybrid classifiers distinguish legitimate and phishing URLs when trained and tested on fixed real data?

RQ2: Can the selected GANs generate valid, unique, varied, and believable phishing-style URLs without copying the real training set?

RQ3: Does adding fixed percentages of generated phishing URLs improve accuracy, precision, recall, F1-score, ROC-AUC, FPR, or FNR compared with the matching real-only model?

RQ4: Do any improvements remain when the models are evaluated on URLs from different sources and on test sets where legitimate URLs greatly outnumber phishing URLs?

RQ5: Can the selected model run locally in a browser extension while keeping URL processing consistent with the trained model and limiting unnecessary navigation blocks?

1.6 Project Approach

The project follows the stages in Figure 1.1. The stages are kept separate so that generated data cannot enter validation or test sets and test results cannot be used to change the training procedure.

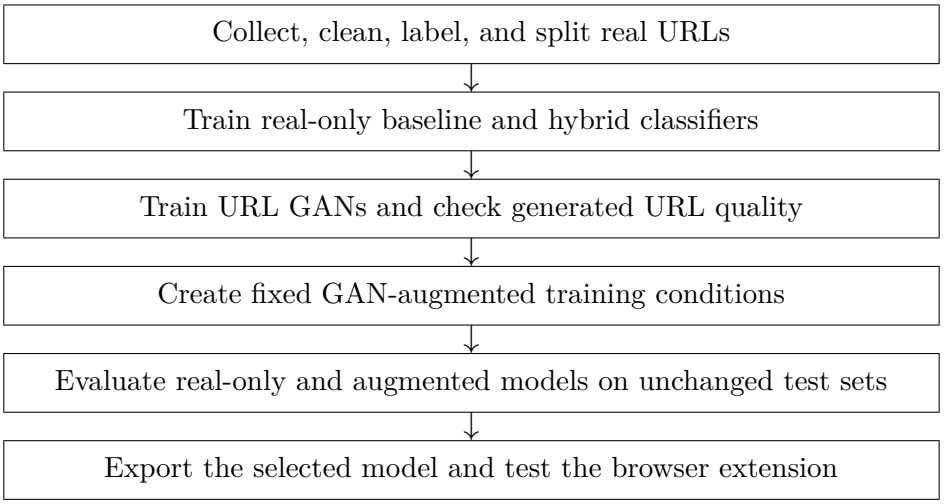


Figure 1.1: High-level workflow of the project.

The main classifier input is the URL text itself. Neural models learn from characters, while traditional models use lexical measurements such as URL length, digit ratio, special-character ratio, subdomain count, path depth, and query structure. The final hybrid model combines a character CNN with a fixed lexical feature vector. Models are trained with repeated random seeds where appropriate, and thresholds are selected using development or calibration data rather than final test labels.

The GAN stage uses phishing URLs from the training data only. Generated candidates must pass validity, duplication, overlap, structure, and diversity checks before they can be added to a classifier training set. Augmented models are then compared with real-only controls using accuracy, precision, recall, F1-score, ROC-AUC, FPR, and FNR.

1.7 Project Scope

The scope is limited to URL-based phishing detection. This keeps the experiments reproducible and allows the selected model to operate locally in a browser.

1.7.1 Included in the Project

- collection and cleaning of labelled legitimate and phishing URLs;
- traditional, neural, and hybrid URL classifiers;

- character-level and lexical URL features;
- GAN training and generated URL quality evaluation;
- controlled real-only and GAN-augmented comparisons;
- internal and external evaluation with complete classification metrics;
- threshold analysis for false-positive and false-negative trade-offs; and
- a local browser extension with warning and passive review behaviour.

1.7.2 Outside the Project Scope

- analysis of email text, attachments, or sender behaviour;
- webpage screenshots, visual similarity, or full Document Object Model analysis;
- automatic submission of visited URLs to a remote monitoring service;
- a claim that generated URLs predict unknown future attack methods;
- commercial deployment or certification as a complete security product; and
- replacement of browser reputation services, DNS security, or human judgement.

1.8 Main Contributions

The project makes the following contributions:

- a controlled comparison of traditional, neural, and hybrid phishing URL classifiers using clearly separated training and evaluation data;
- an evaluation of several GAN approaches that separates generated URL quality from their effect on classifier performance;
- fixed-percentage augmentation experiments that preserve the real training rows and use the same evaluation sets for real-only and augmented conditions;
- an analysis of distribution shift, showing why strong results on an internal test may not transfer to URLs collected from other sources;
- a hybrid model that combines character learning and lexical URL features with multi-source training and score calibration; and
- a working browser extension that performs local inference and separates model warnings from automatic navigation blocking.

1.9 Report Structure

The complete report is planned as follows:

- **Chapter 1 – Introduction** presents the problem, aim, objectives, research questions, scope, approach, and contributions.
- **Chapter 2 – Background** explains phishing URLs, URL structure, classification models, GANs, augmentation, distribution shift, and evaluation metrics.
- **Chapter 3 – Related Work** reviews recent phishing URL detection and GAN-based URL generation research and identifies the gap addressed by this project.
- **Chapter 4 – Methodology and System Design** describes the data, splits, models, GAN designs, quality checks, augmentation rules, and evaluation method.
- **Chapter 5 – Implementation** explains the important software components, training pipeline, GPU use, stored model files, and reproducibility controls.
- **Chapter 6 – Experiments and Results** presents the baseline, GAN-quality, augmentation, external, and threshold results with their analysis.
- **Chapter 7 – Browser Extension and User Testing** describes the browser implementation, warning modes, local data handling, technical tests, and user feedback.
- **Chapter 8 – Conclusion** answers the research questions and summarises the main findings and limitations.
- **Chapter 9 – Future Work** presents practical improvements and research that could continue after this project.

Chapter 2

Technical Background

This chapter introduces the concepts needed to understand the design and evaluation of the project. It begins with phishing and URL structure, then explains URL representations, the classifier families used in the experiments, and the role of generative adversarial networks. The final sections explain why data quality, distribution shift, class balance, thresholds, and complete evaluation metrics are essential when judging a phishing detector.

2.1 Phishing and URL-Based Detection

Phishing is an attempt to impersonate a trusted person or service so that a user performs an unsafe action. The action may be entering credentials, sending a payment, opening a file, or visiting a malicious webpage. A phishing campaign usually combines a delivery method, a deceptive message, a destination URL, and a page designed to appear trustworthy. The URL is therefore only one part of an attack, but it is available early enough to support a warning before the user interacts with the destination.

Software-based phishing detection includes blacklists and whitelists, heuristic rules, webpage-content analysis, visual comparison, network or domain evidence, and machine-learning models. Blacklists are effective for threats that have already been reported, but they cannot contain every newly created or rapidly changed address. Content and visual approaches can provide more evidence, but they require the page to be fetched and may introduce delay or risk. A URL-only model uses the address that is already available to the browser and can run without downloading the full page [5, 6].

This project uses URL-only classification as the primary detector. This choice supports fast local inference and makes the experiments reproducible. However, the model cannot confirm page content, ownership, or later changes. Its output is a risk estimate based on URL patterns, not proof that a website is malicious.

2.2 Structure of a URL

A Uniform Resource Locator identifies a resource and describes how it can be accessed. A simplified URL can contain a scheme, host, optional port, path, query, and fragment. Figure 2.1 shows these components in an example address.

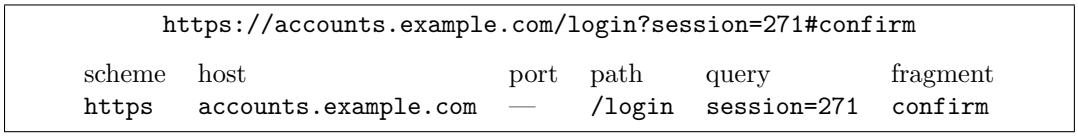


Figure 2.1: Main components of a URL.

The host itself is hierarchical. In `accounts.example.com`, `com` is the top-level domain, `example` is the registered-domain label, and `accounts` is a subdomain. Attackers can place a trusted word in a subdomain or path even when the registered domain is unrelated to the service being imitated. Other suspicious patterns can include an IP address in place of a host name, many nested subdomains, misleading brand tokens, character substitutions, encoded text, unusually deep paths, or a long query string.

None of these patterns proves phishing by itself. Legitimate authentication, advertising, cloud storage, redirection, and tracking services may also produce long and complicated URLs. This is one reason fixed rules often create false positives. Kan and Nguyen Thi showed that URL components, their lengths, orthographic patterns, and token order all contain useful classification information [7]. The detector should learn combinations of evidence rather than treating one feature, such as length, as a final decision.

2.3 Representing URLs for Machine Learning

A learning algorithm requires a numerical representation. This project uses two complementary representations: manually calculated lexical features and a character sequence learned by a neural network.

2.3.1 Lexical Features

Lexical features describe measurable properties of the URL string without visiting the website. Examples include total length, hostname length, numbers of digits, letters, dots and special characters, digit-to-character ratios, subdomain count, path depth, query parameters, and the presence of an IP-style host. Table 2.1 groups the main types.

Table 2.1: Groups of lexical URL features used in phishing detection.

Feature group	Examples	Information represented
Length	URL, host, path, and query length	Overall complexity and unusually long components.
Character counts	Dots, hyphens, digits, symbols, and repeated characters	Surface patterns and possible obfuscation.
Ratios	Digit, letter, and special-character ratios	Composition independent of absolute length.
Host structure	Subdomains, IP-style host, top-level-domain form	How the destination host is organised.
Path and query	Path depth, parameters, delimiters, and encoded characters	Structure after the host and possible redirection or tracking.
Token indicators	Login, account, verify, update, secure, and brand-like terms	Words commonly used in service and credential contexts.

Lexical features are small, fast, and interpretable. They are suitable for Random Forest and Support Vector Machine models, and they can be added to a neural model. Their weakness is that the selected list reflects existing human knowledge. Wang found that both lexical selection and token/vector representations can be useful, while ensemble classifiers performed strongly in the reported comparison [8]. However, an attacker can change a URL in a way that is not well represented by a fixed feature list.

2.3.2 Character-Level Representation

A character-level model first constructs a vocabulary containing the letters, digits, punctuation marks, and special symbols retained during preprocessing. Each character is mapped to an integer. URLs are padded or truncated to a fixed maximum length so that a batch has a consistent shape. An embedding layer then maps each integer to a learned dense vector.

This representation keeps local spelling, punctuation, and order information. It avoids assuming that a URL can be tokenised like an ordinary sentence. Al-Alyan and Al-Ahmadi used this approach with a CNN and a dataset of more than two million URLs, showing that a URL-only character model can operate without predetermined features [6]. Character models still have a maximum input length, so truncation must be reported and long URLs require separate evaluation.

2.3.3 Hybrid Representation

The hybrid representation combines a learned character vector with a lexical feature vector. Before concatenation, numerical lexical values are scaled using parameters fitted on training data only. The combined vector is passed through dense layers to produce a phishing score. The character branch can discover patterns that were not explicitly programmed, while the lexical branch directly provides stable measurements such as length and path depth.

The term *hybrid* in this report refers to this combination of character CNN features and lexical features. It does not mean that the model uses page content, screenshots, or live network requests. To keep model comparisons fair, all classifiers use the same stored real URL rows and labels, even though their representations differ according to the algorithm being tested.

2.4 Classifier Families

The experiments include traditional and neural classifiers so that the effect of GAN augmentation is not judged using one model family only.

2.4.1 Random Forest

A decision tree repeatedly divides samples using feature conditions. A Random Forest trains many trees using sampled rows and subsets of features, then combines their outputs. This reduces the instability of a single tree and works well with mixed nonlinear lexical relationships. It can also provide feature importance estimates. Its main limitation in this setting is that it uses the provided feature vector and does not directly learn the complete order of URL characters.

2.4.2 Support Vector Machine

A Support Vector Machine finds a decision boundary with a large margin between classes. Linear SVMs can be effective with scaled lexical or sparse text features. Kernel variants can represent nonlinear boundaries but may become expensive on very large datasets. Because feature magnitude influences the margin, scaling must be learned from training data and reused unchanged during evaluation and deployment.

2.4.3 Convolutional Neural Network

A one-dimensional CNN moves filters across the embedded character sequence. Each filter can respond to a local pattern such as a token fragment, delimiter combination, or suspicious character sequence. Pooling forms a fixed-size summary and dense layers

produce the final score. CNN operations can be parallelised efficiently on a GPU and are suitable for local URL inference. The URL-only CNN literature supports this design for both ordinary detection and GAN discrimination [5, 6].

2.4.4 RNN and LSTM

A recurrent neural network processes the sequence in order and passes a hidden state from one position to the next. This gives it a direct representation of sequence history, but an ordinary RNN can have difficulty preserving information over long distances. The Long Short-Term Memory network introduces gates that control what is stored, updated, and exposed. LSTMs can therefore model longer dependencies, although their sequential calculation is generally slower than the parallel operations of a CNN.

2.4.5 Hybrid CNN Classifier

The project hybrid feeds the embedded URL to a character CNN and concatenates the resulting representation with scaled lexical features. A simplified calculation is

$$\hat{p} = \sigma(f_{\text{dense}}([f_{\text{CNN}}(x_{\text{char}}), x_{\text{lex}}])), \tag{2.1}$$

where x_{char} is the encoded character sequence, x_{lex} is the lexical feature vector, $[\cdot]$ denotes concatenation, and $\hat{p}$ is the predicted phishing probability or score. The hybrid is not automatically superior: its value must be established using the same training rows and frozen evaluations as the other classifiers.

2.5 Generative Adversarial Networks

2.5.1 Basic Adversarial Training

A GAN contains two networks trained with opposing objectives. The generator G transforms a random vector z into a synthetic sample. The discriminator D receives real and generated samples and learns to distinguish them. In the original formulation, the objective is

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]. \tag{2.2}$$

During alternating training, the discriminator improves its judgement and the generator receives a learning signal that rewards more believable output [9]. The desired result is not a copy of the training set, but new samples that follow useful parts of its distribution.

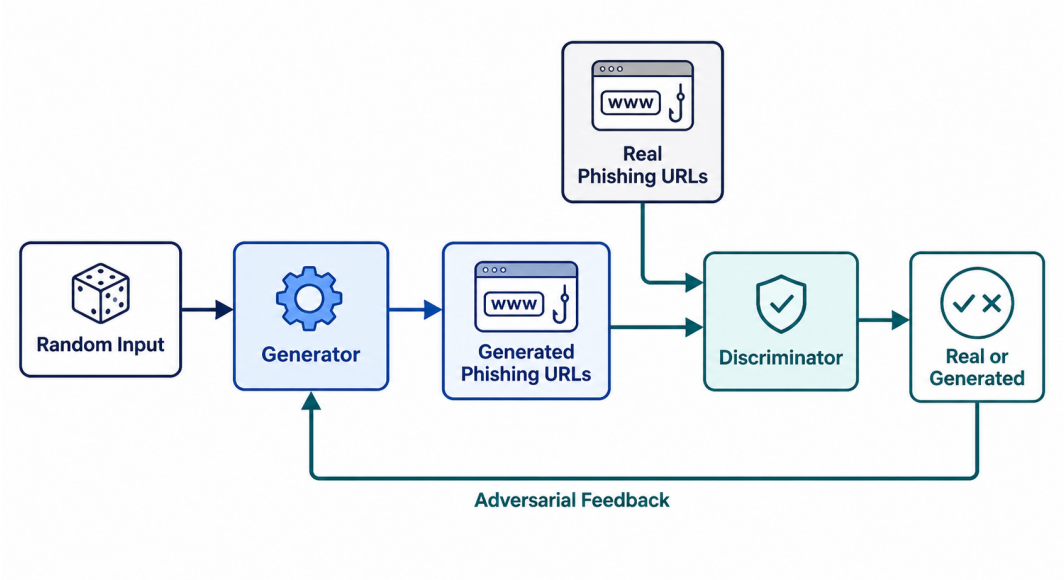


Figure 2.2: Basic adversarial training cycle adapted to phishing URL generation.

GAN training can be unstable. If the discriminator becomes too strong, the generator may receive little useful gradient. If the generator learns only a small number of successful patterns, it may produce many similar samples; this is called mode collapse. Loss values alone are therefore insufficient evidence that generated URLs are useful.

2.5.2 Why URL Text Is Difficult to Generate

Image generators output continuous pixel values, while a URL generator must select discrete characters. Sampling a character breaks ordinary back-propagation because the discrete choice is not differentiable. A sequence generator must also learn syntax: a string may resemble phishing text but still be unusable if its scheme, host, delimiters, or top-level-domain structure is invalid.

The Gumbel–Softmax relaxation approximates a categorical choice using a continuous differentiable vector during training [10]. As the temperature is reduced, its output becomes closer to a one-hot character selection. This provides an end-to-end gradient while retaining discrete characters when URLs are finally sampled.

SeqGAN takes a different approach and treats sequence generation as a policy. The discriminator supplies a reward, and Monte Carlo rollouts estimate the value of incomplete sequences before the final character is produced [11]. This can connect a final realism judgement to earlier character choices, but policy-gradient training may have high variance and is more complex than ordinary supervised sequence learning.

2.5.3 GAN Designs Relevant to the Project

Several supplied studies motivate the GAN designs considered in this project. PDGAN uses an LSTM generator to produce phishing URL sequences and a CNN discriminator to judge URLs [5]. This pairing is appropriate because the generator models ordered text while the discriminator extracts local patterns efficiently. The semi-supervised conditional approach of Kamran et al. conditions generation on URL and class information and gives the discriminator both real-versus-generated and benign-versus-malicious tasks [2].

Wasserstein GAN training replaces the original discriminator interpretation with a critic whose score estimates a distance between real and generated distributions. WGAN-GP adds a gradient penalty to enforce the required smoothness without simple weight clipping [12]. This often improves training stability, but stable losses do not guarantee syntactically valid or diverse URLs.

The comparative phishing-GAN study supplied for this project evaluates basic GAN, DCGAN, WGAN, and SeqGAN models. It reports that generator quality and discriminator classification do not necessarily rank models in the same order, and evaluates generated URLs using structural and classifier-challenge evidence [4]. This directly supports the decision to measure GAN quality separately from downstream classifier accuracy.

2.6 GAN Augmentation for Defensive Training

In this project, a generator is trained only on phishing URLs belonging to the training split. Accepted synthetic phishing URLs are then added to the original real training rows. Validation, calibration, internal test, and external test sets remain real and unchanged. If N_r is the number of real training samples and N_g is the number of accepted generated samples, the generated share of the resulting training set is

$$r_g = \frac{N_g}{N_r + N_g}. \tag{2.3}$$

This definition is important because "10% augmentation" can otherwise mean either ten generated samples per hundred real samples or ten generated samples per hundred total samples. The report states the exact counts for each experiment.

Generated data can help if it adds realistic patterns that are missing or rare in the available phishing data. It can hurt if it contains invalid syntax, repeated templates, generator artifacts, incorrect labels, or structures that do not match real phishing URLs. More synthetic data is therefore not assumed to be better. A decrease in accuracy or F1-score is a valid result and should not be hidden by changing the augmentation percentage after viewing the test set.

2.7 Evaluating Generated URL Quality

Classifier performance alone cannot show what a GAN learned. The generated pool is evaluated before augmentation using the following checks:

Validity: whether the candidate can be parsed and contains an acceptable host and URL-like structure;

Uniqueness: the percentage of generated candidates remaining after exact duplicates are removed;

Novelty: whether a generated URL is absent from the real training, validation, and test data;

Structural realism: whether length, components, delimiters, top-level domain form, and lexical statistics fall within plausible real-data ranges;

Diversity: whether candidates cover different lengths, tokens, hosts, paths, generators, and clusters rather than one repeated template; and

Classifier challenge: whether frozen real-only classifiers find the candidate difficult or place it near their decision boundaries.

A challenging sample is not automatically realistic. A malformed artifact may confuse a classifier precisely because it is outside both real classes. Realism, novelty, and challenge must therefore be considered together. The selection rule must also be frozen before final test evaluation so test labels cannot influence which generated samples are accepted.

2.8 Distribution Shift and Data Leakage

2.8.1 Distribution Shift

Training and test data are identically distributed only under an idealised assumption. In practice, legitimate URLs from top-domain lists differ from URLs observed during normal browsing, and phishing feeds differ in verification rules, collection time, campaign types, and URL lifetime. These differences are called distribution shift. A model may learn the source rather than the class; for example, protocol style or URL length can accidentally identify which feed supplied a sample.

The large URL-only CNN study warns that models should be kept current and notes sampling problems caused by representing legitimate data using only highly ranked sites [6]. For this reason, this project uses both internal and identity-disjoint

external evaluations, including realistic low-phishing scenarios and long legitimate URLs. Strong internal accuracy is not treated as proof of deployment readiness.

GAN augmentation does not predict a future zero-day pattern. It recombines patterns learned from available phishing training data. It may improve coverage of nearby variations and make a classifier less dependent on a few templates, but it cannot guarantee knowledge of a genuinely new attacker behaviour.

2.8.2 Data Leakage

Leakage occurs when information from validation or test data affects training. Examples include duplicate URLs across splits, the same domain identity in training and test sets, fitting a scaler on all rows, selecting a threshold on the final test, or filtering generated samples using final test performance. Leakage produces optimistic measurements that may disappear in real use.

The controlled pipeline therefore deduplicates and checks overlaps, fits preprocessing on training data, trains GANs on training phishing URLs only, selects thresholds on separate development data, and keeps final evaluation sets frozen.

2.9 Evaluation Metrics

For the positive class "phishing", a true positive (TP) is a phishing URL correctly detected; a false positive (FP) is a legitimate URL incorrectly flagged; a true negative (TN) is a legitimate URL correctly accepted; and a false negative (FN) is a phishing URL missed by the detector.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{2.4}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{2.5}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{2.6}$$

$$\text{F1} = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{2.7}$$

$$\text{FPR} = \frac{FP}{FP + TN}, \tag{2.8}$$

$$\text{FNR} = \frac{FN}{FN + TP} = 1 - \text{Recall}. \tag{2.9}$$

Accuracy can look high in a deployment-like dataset where phishing is rare, even if many phishing URLs are missed. Precision indicates how many phishing warnings are correct and is strongly affected by class prevalence. Recall and FNR measure missed

attacks, while FPR measures disruption to legitimate browsing. F1 balances precision and recall but does not include true negatives.

ROC-AUC evaluates how well scores rank positive examples above negative ones across thresholds. It is useful for model comparison, but it does not select an operating threshold or directly describe warning precision in a low-phishing environment. This project consequently reports the complete metric set and the confusion counts rather than presenting accuracy alone.

2.10 Thresholds, Calibration, and Browser Decisions

A binary model normally produces a score and compares it with a threshold t. Lowering t usually detects more phishing URLs and reduces FNR, but also flags more legitimate URLs and increases FPR. Raising t normally has the opposite effect. There is no threshold that independently eliminates both error types.

Calibration tests whether a score corresponds to observed probability. A model can rank URLs well and achieve a strong ROC-AUC while its raw scores remain poorly calibrated. Calibration parameters and thresholds must be estimated on separate development data, then frozen before external evaluation.

The browser extension uses decision bands rather than treating every nonzero score as confirmed phishing. A low-risk result permits ordinary browsing, an intermediate result can display caution or request secondary review, and a high-risk result can display a strong warning according to the selected mode. This policy changes how scores are presented; it does not change the model's underlying ranking ability.

2.11 Chapter Summary

URL-only phishing detection provides fast evidence that is suitable for local browser use, but URL patterns are ambiguous and change across sources. Lexical features provide interpretable structure, character models learn ordered text patterns, and the hybrid model combines both views. GANs can generate additional phishing-style sequences, but discrete text generation is difficult and must be judged using validity, novelty, realism, diversity, and classifier challenge. Controlled augmentation, leakage prevention, external evaluation, and complete metrics are therefore necessary before deciding whether generated data improves detection. Chapter 3 uses this background to compare the supplied research in detail and identify the specific gap addressed by the project.

Chapter 3

Related Work

This chapter reviews research relevant to URL-only phishing detection and GAN-based URL generation. The review concentrates on the papers supplied for the project and on the design decisions that affect the present work. The studies are not ranked by their headline accuracy alone. Their datasets, preprocessing, class balance, split rules, model inputs, and evaluation procedures differ, so a numerical result from one paper is not automatically comparable with a result from another.

3.1 Review Scope and Comparison Method

The reviewed literature is divided into three groups. The first group studies URL structure and traditional machine-learning features. The second group uses character-level neural networks for URL classification. The third group applies GANs to simultaneous detection, synthetic-data augmentation, or adversarial evasion. Review papers were used to confirm the broader classification of phishing-detection methods, but the analysis gives more weight to studies that describe their datasets, architecture, and experimental procedure [13].

Each experimental paper is considered using the following questions:

- What information is given to the model: raw URL characters, lexical features, generated feature vectors, or another representation?
- How many URLs are used, where do they come from, and how are duplicates or repeated hosts handled?
- Are training and testing separated randomly, by time, by identity, or by source?
- What are the generator and discriminator architectures, and what purpose does each serve?

- Are generated URLs evaluated for validity, uniqueness, diversity, and overlap, or only through loss and classifier scores?
- Are accuracy, precision, recall, F1-score, ROC-AUC, FPR, and FNR reported?
- Does the experiment isolate the effect of synthetic augmentation from changes in real data, class balance, thresholds, and test data?

These questions are important because a model can obtain a strong result for the wrong reason. Repeated hosts may allow memorisation, URL schemes may reveal the source, balanced tests may hide poor warning precision in ordinary browsing, and thresholds chosen on the final test set can make performance appear better than it would be on new data.

3.2 URL Features and Traditional Classifiers

Kan and Nguyen Thi studied webpage classification using the URL without fetching the page. Their method segmented addresses and represented URL components, component lengths, orthographic patterns, token sequences, and precedence relations. The study was not specifically a phishing detector, but it demonstrated that the location and order of URL tokens contain useful information and that URL-only features can supplement other representations [7]. This supports the use of hostname, path, query, length, character, and token measurements in the lexical branch of the present hybrid model.

Sahingoz et al. developed a phishing detector from URL data using natural language processing features and several machine-learning algorithms. Their study is important to this project because it treats phishing detection as a URL-text classification problem and provides a traditional baseline for later character-level work [1]. However, later analysis of the dataset found many repeated hosts, illustrating why deduplication and identity separation can materially change reported performance [6].

Wang compared two feature-extraction procedures on 10,000 benign and 10,000 malicious URLs. One procedure used tokenisation and vectorisation, while the other used selected lexical features. Logistic Regression, SVM, Random Forest, Gradient Boosting, and Bagging were evaluated. Token/vector features combined with ensemble methods produced the strongest results; Random Forest achieved 97.35% accuracy and 98.75% AUC in the reported condition [8]. The result supports RF and SVM baselines, but it comes from that paper's own balanced dataset and feature space. It cannot be used as a direct target for an external or low-phishing deployment test.

The traditional literature therefore provides two lessons. First, carefully constructed lexical and token features can be highly informative. Second, the same feature set may

not transfer when data sources or collection periods change. This motivates comparing lexical baselines against neural models on exactly the same real rows in this project.

3.3 Character-Level Deep Learning

Al-Alyan and Al-Ahmadi proposed PUCNN, a URL-only character CNN, and created the Massive URL Phishing Detection dataset. After removing duplicate URLs and repeated hosts, their main dataset contained 1,140,599 legitimate and 1,167,201 phishing URLs. The CNN used a maximum length of 256 characters, an alphabet of 69 symbols, a character embedding, one-dimensional convolution, global max pooling, and dense layers [6].

PUCNN achieved 95.78% accuracy with a random split. More importantly, the authors also ordered the phishing data by collection time: older URLs were used for training and newer URLs for testing. Accuracy fell to 88.54%, a decrease of about 7.5 percentage points. The paper also showed that a simple HTTP/HTTPS scheme rule could reach approximately 74% accuracy on its collected data, so the scheme was removed to reduce source-related bias. These experiments are particularly relevant because they show that random internal evaluation can overstate performance when phishing patterns change over time.

The character-CNN study supports the main neural input used in this project, but it also exposes two limitations that the present experiments must address. A fixed maximum length truncates some information, and even a model trained on more than two million URLs can experience substantial temporal distribution shift. A larger dataset is helpful, but size alone does not guarantee generalisation.

3.4 GAN-Based Phishing Research

GAN research in phishing has followed several different objectives. Some studies use the discriminator as the final detector. Some add generated URLs to classifier training. Others deliberately search for adversarial samples that evade a black-box detector. These objectives should not be treated as the same experiment.

3.4.1 PWDGAN: Black-Box Evasion

PWDGAN was designed to generate adversarial malicious examples that bypass black-box phishing classifiers. It used 10,017 malicious websites from PhishTank and 11,300 benign websites from Alexa. URLs were converted to 150 numeric features. The generator contained dense hidden layers, while the discriminator learned to imitate the output of an external phishing detector and provide feedback to the generator [3].

The study attacked SVM, Decision Tree, Random Forest, Logistic Regression, and MLP detectors. After 500 epochs, the reported true-positive rate for most of the attacked classifiers fell to approximately zero on adversarial examples. This is evidence that a detector can be vulnerable to synthetic feature perturbations. It is not evidence that the generated examples are valid, human-readable phishing URLs, nor does it show that adding those examples to a fixed classifier training set improves external real-URL detection. PWDGAN is therefore most useful to this project as motivation for classifier-challenge testing and adversarial hard-sample selection.

3.4.2 Conditional GAN with Joint Detection

Kamran et al. proposed a semi-supervised conditional GAN that simultaneously generates URLs and classifies them. The generator combines an original URL, class label, and noise vector. Its convolutional encoder–decoder produces a synthetic URL representation. The multi-headed discriminator predicts both real-versus-generated status and benign-versus-malicious class. Reconstruction, adversarial, and classification losses are combined during training [2].

The study sampled 50,000 balanced URLs of at most 200 characters, using 40,000 for training and 10,000 for testing. The model was trained for 200 epochs. It reported 95.52% accuracy, 95.08% precision, 96.00% sensitivity, 95.54% F1-score, and 95.52% AUC on real URL classification. On a balanced real and adversarial evaluation, the discriminator achieved 87.45% accuracy and detected 74.9% of the adversarial examples.

The conditional design is useful because generation and classification share class information, and reconstruction can keep generated output close to an input URL. At the same time, high similarity may mean the generator makes local variations rather than exploring broad new phishing modes. The study reports structural similarity, but does not provide the complete uniqueness, exact training-overlap, and cross-source tests required by the present project.

3.4.3 PDGAN: LSTM Generator and CNN Discriminator

PDGAN uses an LSTM generator and a nine-layer CNN discriminator. The generator learns ordered URL characters and the discriminator extracts local character patterns. It was evaluated using the large MUPD dataset, which after preprocessing contained 1,167,201 phishing and 1,140,599 legitimate URLs. The authors removed duplicate URLs and repeated hosts before splitting the data [5].

The reported final detector achieved 97.58% accuracy and 98.02% precision. Experiments added generated phishing sets of different sizes; the paper reports its strongest condition after adding 50,000 synthetic URLs. The work is close to the objective of

this project because it combines a sequence generator, character CNN, and defensive augmentation. It also acknowledges that some generated URL details have incorrect semantic information. This reinforces the need to inspect generated samples and to report quality separately from the discriminator's classification metrics.

PDGAN primarily evaluates its own GAN discriminator and uses its own split. It does not provide a paired, multi-seed comparison across RF, SVM, LSTM, CNN, RNN, and a lexical–character hybrid on unchanged external tests. The present work extends the experimental focus rather than attempting to reproduce its headline result.

3.4.4 WGAN-GP Augmentation

Pham et al. applied WGAN-GP to character-level malicious URL generation and then trained LSTM and GRU classifiers with and without generated phishing URLs [14]. The study used malicious URL sources and two Common Crawl legitimate sources. It evaluated total datasets of 150,000, 300,000, 600,000, and 1.5 million samples. The generator used one-dimensional convolutions and residual blocks, while the critic used Wasserstein distance and a gradient penalty.

The reported effect depended on both dataset size and classifier. At 150,000 samples, LSTM accuracy increased from 96.08% to 96.47% with augmentation. At 600,000 samples it decreased from 97.28% to 97.16%, and at 1.5 million it decreased from 97.67% to 97.57%. GRU performance decreased slightly in every reported augmented condition. The authors concluded that generated data was more useful when real training data was limited.

This is important evidence against assuming that augmentation must always improve a classifier. It supports optional real-data-size experiments, but also shows why the project should not change dataset size merely to find a winning GAN configuration.

3.4.5 Comparison of Basic GAN, DCGAN, WGAN, and SeqGAN

Pham, Pham, and Ta compared basic GAN, DCGAN, WGAN, and SeqGAN on a dataset containing two million legitimate and two million phishing URLs. One million phishing URLs were used for discriminator training, and a test set contained one million phishing and one million legitimate URLs. A second experiment trained GANs on balanced real data and added generated URLs to RF and LSTM training [4].

The discriminator results differed from generator-quality results. Basic GAN, DCGAN, WGAN, and SeqGAN obtained reported discriminator accuracies of 99.98%, 99.88%, 98.99%, and 85.05%, respectively. However, only about half of the basic-GAN, DCGAN, and WGAN outputs passed the top-level-domain structural check, whereas SeqGAN reached 92.22%. The paper's examples also show severe malformed

character output from some continuous GAN variants. This demonstrates that a strong discriminator does not prove that its generator produces good URLs.

The downstream augmentation effects were small and mixed. RF accuracy was 90.46% on original balanced data and ranged from 90.28% to 90.85% after the different GAN additions. LSTM accuracy was 96.45% on original data and ranged from 96.16% to 96.53% after augmentation. SeqGAN produced the strongest top-level-domain validity and the largest RF reduction, while basic GAN produced the largest LSTM reduction. In that paper these reductions were interpreted as evidence of classifier deception; for defensive augmentation, the meaning is different and must be tested on real held-out data.

One methodological caution is that the paper describes searching thresholds for the best accuracy on the test set. A final deployment study should instead select thresholds on validation or calibration data and use the test set once. The project follows the latter procedure.

3.4.6 Generative Round Network

Wanda et al. proposed the Generative Round Network, which adds stochastic rounding to the generative components. It used 14,000 records with an 80/20 train–test split. The reported GRN result was 95% accuracy, 97% precision, 97% recall, and 97% F1-score, compared with 94% accuracy for the paper's standard GAN condition [15].

The stochastic mechanism is intended to keep generator and discriminator training balanced. The study analyses generator and discriminator losses, but loss balance is only indirect evidence of URL realism and diversity. The dataset is relatively small compared with the million-scale URL studies, and the authors identify temporal datasets and further architectures as future work. Its results are therefore useful as evidence that alternative discrete operations may stabilise training, not as a direct comparison with this project's models.

3.5 Comparison of the Reviewed Studies

Table 3.1 summarises the studies most closely related to the implemented system. Reported metrics are reproduced to describe each paper; they should not be read as a common leaderboard.

Table 3.1: Comparison of closely related URL-detection and GAN studies.

Study	Model and data	Main reported result	Relevance and limitation
Al-Alyan and Al-Ahmadi [6]	Character CNN; about 2.31M deduplicated legitimate and phishing URLs.	95.78% random-split accuracy; 88.54% with date split.	Strong evidence for URL characters and temporal shift; no GAN augmentation.
Wang [8]	RF, SVM, LR, GB, and Bagging; 20K balanced URLs; lexical and token/vector features.	RF with token/vector features: 97.35% accuracy and 98.75% AUC.	Supports traditional baselines; smaller balanced evaluation.
Conditional GAN [2]	Convolutional conditional generator and multi-task discriminator; 50K balanced URLs.	95.52% real-URL accuracy; 87.45% real/adversarial accuracy.	Joint generation and classification; limited cross-source and diversity evidence.
PDGAN [5]	LSTM generator and CNN discriminator; about 2.31M URLs after preprocessing.	97.58% accuracy and 98.02% precision.	Closest defensive GAN design; quality analysis and paired cross-model testing remain limited.
WGAN-GP augmentation [14]	Convolutional WGAN-GP plus LSTM and GRU; 150K to 1.5M conditions.	Small LSTM gain at 150K; mixed or negative effects at larger sizes and for GRU.	Shows that augmentation benefit depends on real-data size and classifier.
Four-GAN evaluation [4]	Basic GAN, DCGAN, WGAN, and SeqGAN; 4M real URLs.	Discriminator and generator rankings differed; augmentation changes were small and mixed.	Strong motivation for separate quality tests; test-based threshold search is a limitation.

Continued on next page

Table 3.1 – continued

Study	Model and data	Main reported result	Relevance and limitation
PWDGAN [3]	Dense feature-space generator and surrogate discriminator; 21,317 URLs.	Most attacked classifiers approached 0% TPR after 500 epochs.	Demonstrates adversarial weakness; does not establish useful, valid URL-text augmentation.
GRN [15]	GAN with stochastic rounding; 14K URLs.	95% accuracy and 97% F1-score.	Explores training balance; smaller random split and limited direct quality evidence.

Several patterns emerge from the comparison. Large random-split datasets often produce strong internal results, but the temporal CNN experiment shows that performance can fall sharply on later phishing URLs. Generator and discriminator quality are separate: the best discriminator may belong to a GAN that produces malformed strings. Augmentation effects can also be classifier- and data-size-dependent. Finally, accuracy is reported more consistently than FPR, FNR, external precision, uniqueness, or overlap, although those measures are central to browser use.

3.6 Research Gap

The reviewed work demonstrates that URL-only models and GAN-based approaches are promising, but it leaves a gap between a high internal score and evidence that synthetic phishing URLs improve a practical detector. The main missing elements are:

- a single frozen real training dataset shared by every compared classifier;
- paired real-only and GAN-augmented conditions that change only the added synthetic phishing rows;
- several fixed augmentation percentages retained even when they do not improve performance;
- repeated seeds and reporting of variation rather than one favourable run;
- explicit generated-URL validity, uniqueness, novelty, overlap, structural realism, diversity, and classifier-challenge measurements;
- internal, identity-disjoint, source-diverse, temporal or recently collected, low-phishing, and long-legitimate-URL evaluations;

- complete reporting of precision, recall, F1-score, ROC-AUC, FPR, FNR, and confusion counts; and
- a browser decision policy whose threshold is selected without using the final test labels.

The project addresses this gap using a controlled experiment rather than assuming that every synthetic sample is beneficial. Real-only models establish the reference performance. GANs are trained only on phishing URLs from the training split. Generated candidates pass fixed quality and overlap checks, then the same accepted pool is supplied to the classifiers being compared. Validation and test sets remain real and unchanged. Results are interpreted as evidence of benefit, no effect, or degradation according to the measured data.

The hybrid classifier adds a further comparison not covered consistently in the supplied GAN papers: character CNN features and lexical measurements are combined in one detector, but the hybrid receives the same real URL identities as the ordinary baselines. Weighted sampling is treated as a separate training condition rather than as additional data. This distinction prevents the hybrid from gaining hidden source access.

3.7 Chapter Summary

Prior research establishes that URL structure, lexical features, RF and SVM models, and character-level neural networks can all support phishing detection. GAN studies show that LSTM, convolutional, conditional, Wasserstein, sequential, and stochastic-rounding designs can contribute to generation or discrimination. They also show that GAN output may be malformed, that a strong discriminator does not guarantee a strong generator, and that augmentation may help, have no effect, or reduce performance.

The main opportunity is therefore methodological: compare real-only and GAN-augmented classifiers under identical data and evaluation conditions, while measuring generated URL quality separately. Chapter 4 describes how that controlled methodology and the complete system were designed.

Chapter 4

Methodology and System Design

This chapter explains the main research steps followed in the project and the design of the final system. It covers how the data were prepared, how the classifiers and GANs were trained, how synthetic URLs were selected, and how the models were evaluated and deployed. Its purpose is to make the experimental procedure clear and reproducible; individual source-code functions and programming instructions are outside the scope of this chapter.

4.1 Research Design

The project used an experimental comparison design. A phishing URL was treated as the positive class and a legitimate URL as the negative class. The main question was whether adding GAN-generated phishing URLs to a fixed real training set changed detection performance on data that was not used for training or model selection.

The central comparison was therefore

$$\text{fixed real training data} \quad \text{against} \quad \text{the same real data plus synthetic phishing URLs.} \tag{4.1}$$

The validation and test sets, model family, random seeds, preprocessing, threshold rule, and evaluation metrics were held fixed within each controlled experiment. A synthetic condition was not changed simply because it failed to beat the real-only control. No improvement, or a small reduction, was retained as a valid result.

The work used three levels of evidence:

1. **Internal evaluation** measured performance on a frozen, hostname-separated test drawn from the same main source as training.

2. **External evaluation** measured transfer to separately collected legitimate and phishing URLs with no accepted identity overlap.

3. **Browser evaluation** tested the exported model, decision policy, inference reliability, and voluntary user feedback in the working extension.

The design deliberately separates these levels. High internal performance does not by itself prove that a model transfers to a new URL source, and a working browser interface does not by itself prove production readiness.

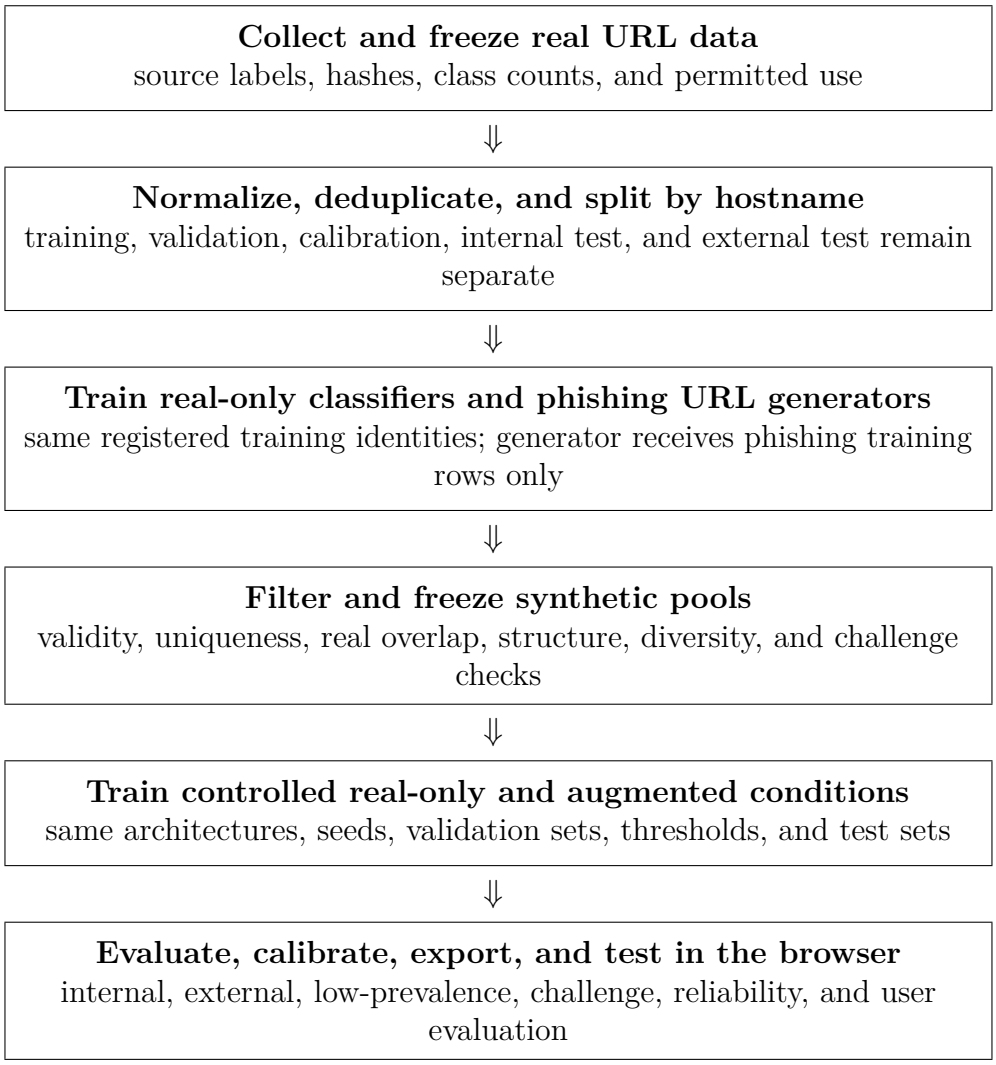


Figure 4.1: Overall experimental workflow.

4.2 Development Through Three Versions

The final methodology was reached through three project versions. These versions are not treated as interchangeable repetitions because their datasets and controls differ. They document how the research problem became more precise.

Version 1 is retained as feasibility evidence, and Version 2 is retained as the large controlled one-dataset study. Version 3 supplies the final system and the main

Table 4.1: Purpose of the three project versions.

Version	Main purpose	Methodological lesson
1	Build the first complete baseline, GAN, augmentation, and outside-testing pipeline using the large original URL dataset.	Strong internal results and unique generator outputs were insufficient evidence of external transfer.
2	Use one frozen real dataset for controlled real-only and 10–40% augmentation comparisons across seeds.	Synthetic difficulty and diversity had to be studied separately from real-test accuracy; source shift remained a major problem.
3	Diagnose source shift, introduce multi-source real coverage, train the hybrid system, improve the GAN design, calibrate thresholds, and export the browser model.	Fair data access, source-aware validation, low-prevalence testing, and deployment policy were as important as the classifier architecture.

deployment-oriented evaluation. Earlier results are reported with their original conditions and are not merged into a single statistical average.

4.3 Data Sources and Dataset Roles

4.3.1 Source separation

The project uses legitimate and phishing URLs from more than one source. Each source is assigned a stated role: training, validation, calibration, internal testing, external testing, or browser-user evaluation. A URL collected for a final test is not moved into training after its result is inspected.

The original Version 3 master file, `URLs_New.csv`, contained 450,176 input rows. Cleaning and hostname-separated partitioning produced the frozen original splits in Table 4.2. The generator contract allowed only the 82,219 phishing URLs in the training split. Validation and test phishing URLs were never supplied to a generator.

Table 4.2: Frozen original Version 3 splits.

Split	Legitimate	Phishing	Total
Training	276,400	82,219	358,619
Validation	29,535	9,948	39,483
Internal test	38,908	10,321	49,229

External transfer analysis later produced a multi-source stored training pool of 363,692 real URLs. It contained 276,400 original legitimate URLs, 3,012 outside legitimate coverage URLs, 82,219 original phishing URLs, 2,000 PhishTank URLs,

and 61 OpenPhish URLs. Identities previously frozen for external evaluation were excluded from the added training material.

The initial SeqGAN study used the 82,219 original phishing training rows, whereas MS-HardGAN-C used all 84,280 phishing rows in the multi-source pool. This difference was appropriate for their original development stages, but it could confound a direct comparison between the two generators. A later matched generator experiment therefore retrained SeqGAN using the exact 84,280-row phishing token file used by MS-HardGAN-C. File hashes confirmed byte-for-byte identity. Each generator retained its registered architecture and training objective; the downstream comparison then used the same 363,692 real HYB-MS rows, 40,410 generated rows, three classifier seeds, calibration data, and frozen tests. This experiment compares complete generator designs while controlling their real phishing input.

A final registered update added 1,746 legitimate and 1,746 phishing URLs to both final training conditions. Consequently, the final real-only file had 367,184 real URLs. The final browser candidate added 40,410 accepted synthetic phishing URLs, producing 407,594 stored training rows. The two conditions therefore differed only by the synthetic component.

Table 4.3: Principal datasets and their permitted roles.

Dataset stage	Legit.	Real phish.	Synthetic	Role
Original V3 training	276,400	82,219	0	Baselines and generator input
Original V3 validation	29,535	9,948	0	Early stopping and initial thresholds
Original V3 internal test	38,908	10,321	0	Frozen internal evaluation
Multi-source real pool	279,412	84,280	0	MS and hybrid development
Matched generator input	0	84,280	0	SeqGAN/MS-HardGAN-C input control
Final updated real-only	281,158	86,026	0	Final real control
Final updated GAN condition	281,158	86,026	40,410	Final augmented model
Browser calibration cohort	3,800	200	0	Threshold selection only
Final browser test	5,700	300	0	Frozen low-prevalence test
Later external test	9,700	300	0	Post-freeze evaluation

4.3.2 Label convention

The project uses $y = 0$ for legitimate and $y = 1$ for phishing. Incoming files were not assumed to share this convention. Their labels were mapped explicitly before concatenation. This was particularly important for `new_data_urls.csv`, in which the source convention used 1 for legitimate. Class counts were checked after every mapping so that a reversed source could not silently enter training.

4.3.3 Illustrative real URL records

Table 4.4 gives a small set of records from the stored project data. These examples are included only to make the two labels concrete; they were not selected to demonstrate model performance. Phishing addresses are defanged and identifying path tokens are shortened so that they cannot be followed directly from the report. Their labels come from the stated data source rather than from visual judgement by the author [16].

Table 4.4: Illustrative legitimate and phishing URL records used by the project.

Label	Defanged or non-clickable example	Stored source and role
Legitimate	`https://www.google.com/`	`URLs_New`; training
Legitimate	`https://www.wikipedia.org/`	`URLs_New`; training
Legitimate	`https://ros.org/`	Tranco; external test
Phishing	`hxxps://www[.]dpdlocoug[.]top/com`	PhishTank; external test
Phishing	`hxxp://allegrolokalnie[.]lokalna-043289[.]shop`	PhishTank; external test
Phishing	`hxxps://docs[.]google[.]com/forms/d/e/[redacted]/viewform`	PhishTank; external test

4.4 URL Preparation and Leakage Control

4.4.1 Normalization

Each raw URL passed through the same deterministic preprocessing procedure. The main operations were:

1. remove surrounding whitespace and reject empty values;
2. accept HTTP(S) URL structure and supply a parsing scheme when needed;

3. normalize the hostname to lower case and remove user information;
4. convert internationalized hostname components consistently;
5. preserve meaningful path and query structure;
6. remove the scheme from the model string; and
7. produce one canonical string for duplicate and overlap checks.

The scheme was removed after an audit showed that HTTP and HTTPS exposure could act as a source shortcut. The model was intended to learn URL structure rather than treat the presence of HTTPS as proof of legitimacy.

4.4.2 Duplicate and conflict removal

Duplicates were checked at several levels: raw normalized string, canonical URL, encoded character sequence, hostname, and registered domain when appropriate. Exact duplicates were removed. If the same canonical identity appeared with conflicting labels, it was excluded from the controlled split and recorded for inspection rather than being assigned an arbitrary class.

Generated URLs were also checked against all permitted real training rows. An accepted synthetic pool could not contain exact duplicates, accepted real overlap, or repeated generator templates above the frozen limit. These checks distinguish novelty from simple memorisation.

4.4.3 Hostname-separated splitting

Random row splitting can place two paths from the same host in both training and testing. The model may then recognise the host instead of learning transferable phishing structure. To reduce this leakage, the main partitions were assigned by hostname using a deterministic seed-42 hash. All URLs associated with one hostname were placed in only one of training, validation, or test.

After splitting, automated assertions checked that the hostname intersections were empty. Dataset counts and cryptographic hashes were stored with the artifacts so rerunning a model could not silently use a different file.

4.5 Input Representations

4.5.1 Character representation

The neural sequence models use a 168-symbol vocabulary. Each standardized URL is encoded as integer character identifiers and padded or truncated to 128 characters. An

unknown-character symbol handles characters outside the fixed vocabulary. Padding positions are excluded where the architecture supports masking.

The 128-character limit keeps GPU memory and browser inference predictable. Long URLs are not completely ignored by the hybrid because its lexical branch still measures full URL, hostname, path, and query lengths. Nevertheless, characters after position 128 are unavailable to the character branch and this is treated as a stated limitation.

4.5.2 Lexical representation

The conventional RF and SVM experiments used URL-only lexical measurements. The final hybrid used 35 scheme-neutral, Public-Suffix-List-aware features. They covered:

- total URL, hostname, registered-domain, subdomain, path, query, and fragment lengths;
- counts or ratios of digits, letters, dots, hyphens, slashes, underscores, percent signs, equal signs, ampersands, and other delimiters;
- subdomain depth, path depth, query parameter count, and hostname-token structure;
- IP-literal, punycode, port, and unusual-host indicators;
- character diversity and entropy-like measurements; and
- indicators for URL tokens commonly associated with account, login, verification, payment, or security wording.

Scaling statistics were fitted on training data only. Validation, calibration, internal-test, and external-test rows did not influence the feature medians or interquartile ranges.

4.6 Classifier Design

Five baseline families were retained because they represent different levels of complexity. RF and linear SVM provide interpretable lexical references. RNN and LSTM test sequential character learning. CNN tests local character-pattern learning. The hybrid combines character and lexical evidence.

Table 4.5: Classifier configurations used in Version 3.

Model	Input	Main configuration
RF	Lexical features	300 trees, maximum depth 32, minimum leaf size 2, square-root feature sampling
Linear SVM	Scaled lexical features	$C = 1.0$, maximum 20,000 optimisation iterations
RNN	Character sequence	48-dimensional embedding, 96 hidden units, one tanh recurrent layer
LSTM	Character sequence	48-dimensional embedding, 96 hidden units, gated recurrent representation
CNN	Character sequence	48-dimensional embedding; 128 channels for kernels 3, 4, and 5; global maximum pooling
HYB-MS	Character sequence and 35 lexical features	CNN character branch, two-layer lexical branch, fused hidden layer, dropout, source-aware training

4.6.1 Fairness of classifier comparisons

Fairness does not require every classifier to have the same architecture or feature representation; that would remove the reason for comparing model families. It requires that the compared conditions have equal access to the registered real URL identities, use the same labels and evaluation rows, and do not receive hidden test information.

Accordingly, models in each controlled comparison used the same permitted real training rows. RF and SVM received the lexical representation suited to traditional methods, while sequence models received characters. The hybrid's use of both representations was declared as part of its architecture and was compared against a real-only hybrid of identical architecture. In the fair GAN experiment, the same accepted synthetic URL identities were supplied to every classifier condition at the stated ratio.

4.6.2 Hybrid multi-source model

HYB-MS contains two branches. The character branch embeds the URL and applies one-dimensional convolutional filters with widths 3, 4, and 5. Global maximum pooling produces a fixed representation of local character patterns. The lexical branch passes the 35 scaled features through two fully connected ReLU layers. Both outputs are concatenated, passed through a fused hidden layer and dropout, and mapped to one phishing logit.

The original HYB-MS training design also used weighted source sampling. The 363,692 stored real URLs remained unchanged, but sampling with replacement controlled how often small outside sources were presented during an epoch. This changed exposure frequency, not the number of unique stored URLs. Natural sampling and

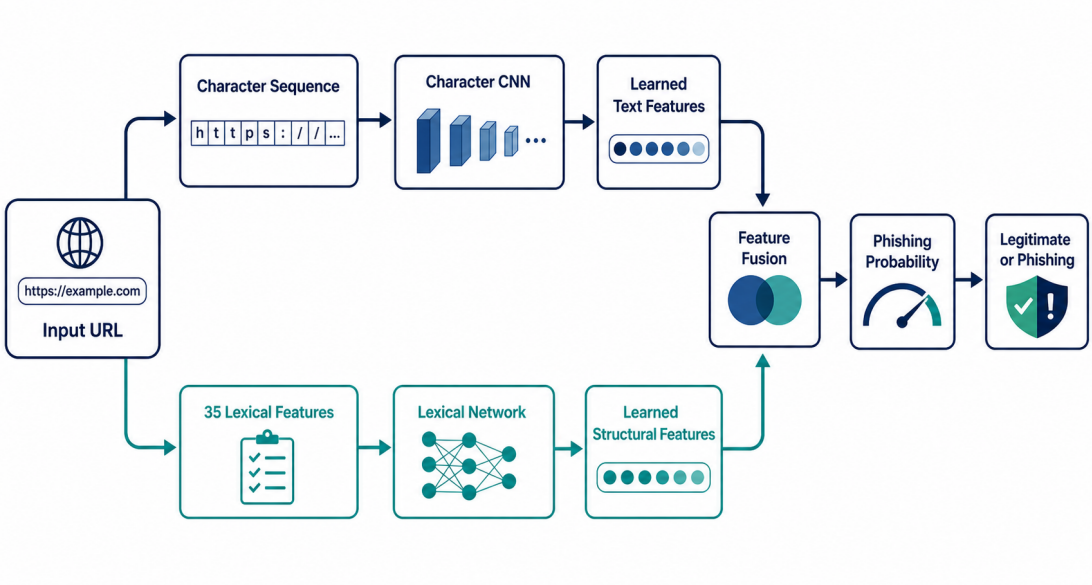


Figure 4.2: High-level architecture of the hybrid URL classifier.

weighted sampling are therefore reported as different conditions.

Class-conditional CORAL alignment was tested to reduce source-specific feature differences. Legitimate representations were aligned only with legitimate representations, and phishing representations only with phishing representations. The registered CORAL weight was 0.01. This avoided forcing opposite classes toward the same distribution.

4.7 GAN Design and Training

4.7.1 Training corpus

The initial Version 3 generators were trained on the 82,219 phishing URLs from the original training split. Multi-source GAN development used 84,280 real phishing URLs: 82,219 original URLs, 2,000 PhishTank URLs, and 61 OpenPhish URLs. Generator validation and final classifier test rows were excluded.

Every text generator first learned next-character prediction by maximum likelihood. Adversarial training followed only after this stage. This is needed because direct adversarial learning over discrete characters is unstable; SeqGAN addresses the problem through policy gradients [11], while the other implemented text generators use a differentiable Gumbel-Softmax approximation [10].

4.7.2 Implemented generators

SeqGAN was retained because it is designed for discrete sequence generation. RelGAN was implemented as a relational alternative, but its structural output required

Table 4.6: Implemented Version 3 GAN configurations.

GAN	Generator	Discriminator	Training schedule
SeqGAN	Autoregressive character LSTM trained as a stochastic policy	Character CNN reward model	5 MLE + 3 adversarial epochs; 100 adversarial steps/epoch
RelGAN	Relational-memory generator with four memory slots and four attention heads	Multi-representation character CNN	5 MLE + 3 adversarial epochs; 150 adversarial steps/epoch
URL-GAN	Two-layer character LSTM with Gumbel-Softmax output	Character CNN with kernels 2, 3, 4, and 5	5 MLE + 4 adversarial epochs; 150 adversarial steps/epoch
MS-HardGAN-C	Conditional two-layer LSTM using structure, source, and latent inputs	Multi-head character CNN for authenticity, pattern, and source	5 MLE + 4 adversarial epochs; 250 adversarial steps/epoch

particularly strict filtering. URL-GAN provided a simpler name and architecture for the two-layer LSTM/CNN Gumbel design. MS-HardGAN-C was the final source- and pattern-conditioned generator used to construct the synthetic pool for the browser candidate.

4.7.3 Classifier-challenging objective

MS-HardGAN-C was designed to generate phishing-style strings that were not merely easy copies of the training corpus. Frozen CNN, LSTM, and RNN classifiers scored differentiable candidates. Their raw scores were first converted to validation-based percentiles because the score scales of different model families are not directly comparable. The mean challenge signal rewarded candidates that multiple classifiers found difficult.

The challenge weight was 0.05 and began in adversarial epoch 3. It was evaluated every fourth generator step. Realism, pattern classification, source classification, maximum-likelihood regularisation, feature matching, and diversity remained active. Thus, a malformed string could not succeed only by receiving a low phishing score.

This procedure creates difficult stress examples based on known evidence. It does not predict an unknown future attack concept. In the report these outputs are described as challenging or zero-day-like samples, not as forecasts of actual zero-day URLs.

4.7.4 Epoch interpretation

The final generator completed five full MLE epochs and four adversarial epochs with 250 steps each, giving 1,000 adversarial update cycles. The neural classifiers had a maximum of eight epochs and an early-stopping patience of two. CNN, LSTM, and RNN baseline runs executed eight epochs, while the retained checkpoint varied by seed.

The final augmented hybrid runs stopped after 3, 4, and 3 epochs for seeds 42, 43, and 44 and retained epochs 1, 2, and 1, respectively. Each epoch contained hundreds of thousands of URL presentations. Stopping early therefore reflected the registered validation rule rather than an attempt to reduce training work. RF and SVM are not epoch-based: RF used 300 trees and SVM used iterative convex optimisation with a maximum of 20,000 iterations.

4.8 Synthetic URL Quality Control

Generated data were evaluated before they were used for augmentation. The quality procedure included:

1. syntactic parsing and permitted-character checks;
2. plausible hostname and top-level-domain-like structure;
3. rejection of empty hosts, abnormal separators, repeated protocols, excessive padding, and other structural outliers;
4. exact duplicate removal;
5. exact and canonical overlap checks against real data;
6. repeated-template and near-duplicate inspection;
7. URL-length, token, path, query, and lexical-distribution comparison;
8. uniqueness, acceptance, and memorisation measurements; and
9. frozen-classifier challenge and held-out synthetic-challenge tests.

4.8.1 Illustrative generated samples

Table 4.7 shows compact examples taken from the frozen MS-HardGAN-C 10% pool. They are synthetic phishing candidates, not confirmed live attacks and not predictions of future incidents. The displayed strings are defanged; query values are shortened where necessary. They illustrate the mixture of subdomains, path depth, digits,

separators, and unusual character sequences learned by the generator. These few examples are for interpretation only; the quantitative quality measures were calculated on the full accepted pool.

Table 4.7: Illustrative defanged MS-HardGAN-C samples from the frozen 10% pool.

Pattern ID	Generated URL text
0	`wwww[.]isfiontt0[.]com/wpatufin/tepeb/`
1	`wwwl[.]anferjalbi[.]cige[.]com/?eev=[redacted]`
2	`jilitebap[.]com/1/drrgg`
3	`ukuoll-seryin[.]com/wwfposidimatexej[.]cum/arinde/dalie[.]chl`
4	`megomlqr[.]com/wtuned/modir/ocoliraide/reglfinex20023[.]50`
7	`mlobacy[.]co/mlsrirud`

Uniqueness alone was not treated as proof of quality. Random malformed strings can be perfectly unique. The accepted pool therefore had to be novel, valid, structurally plausible, sufficiently diverse, and free from permitted real-data overlap.

The realism-filtered condition applied the frozen structural and lexical rules. The realism-filtered plus challenging condition additionally preferred URLs that several calibrated classifier families found difficult. Diversity caps prevented selection from collapsing onto one generator, template, or lexical cluster. Selection rules were frozen before final test labels were viewed.

4.9 Augmentation Conditions

Let N_r be the number of fixed real training URLs and let r be the desired synthetic fraction of the final augmented dataset. The number of synthetic URLs is

$$N_g = \text{round}\left(\frac{r}{1-r} N_r\right). \tag{4.2}$$

This definition means that a 10% condition contains 10% synthetic rows in the combined file, rather than simply adding 10% of the original row count.

Table 4.8: Registered augmentation counts for the 363,692-row multi-source real pool.

Synthetic fraction	Synthetic URLs	Combined rows
10%	40,410	404,102
20%	90,923	454,615
30%	155,868	519,560
40%	242,461	606,153

The synthetic pools were nested where possible so that a larger condition extended the smaller pool rather than replacing it with an unrelated sample. The real rows

were preserved. Augmentation ratios were not changed after observing test results to search for a winning configuration.

One additional controlled 10% experiment compared four selection conditions:

1. real-only;
2. random accepted GAN augmentation;
3. realism-filtered random augmentation; and
4. realism-filtered plus classifier-challenging augmentation.

All four conditions used the same real identities, synthetic count, seeds, model settings, validation rows, thresholds, and test rows. Generator composition was recorded so generator identity did not become a hidden variable.

4.10 Training and Model Selection

Neural training used the NVIDIA RTX 3060 Laptop GPU, CUDA, mixed precision, AdamW, gradient clipping, and deterministic seeds 42, 43, and 44. The maximum classifier training length was eight epochs with early-stopping patience of two. Class-weighted binary cross-entropy reduced the effect of class imbalance.

Checkpoints were selected using validation performance. For the source-aware hybrid, the selection score combined the frozen original validation set and a separate cross-source domain-validation set. This prevented a checkpoint from being selected solely because it improved the familiar source while becoming worse on the outside source.

Operating thresholds were selected on validation or calibration data. The primary rule maximized the registered objective subject to the stated FPR cap. The final test labels were not used to choose epochs, select synthetic samples, calibrate the score threshold, or modify the browser decision tiers.

The three seed checkpoints were retained independently. For final ensemble inference, their sigmoid scores were averaged:

$$\bar{p}(u) = \frac{p_{42}(u) + p_{43}(u) + p_{44}(u)}{3}. \tag{4.3}$$

The browser uses the same three exported ONNX models. A parity test compared PyTorch and ONNX outputs before the browser bundle was accepted.

4.11 Evaluation Protocol

4.11.1 Classifier metrics

The main labelled-dataset metrics were accuracy, precision, recall, F1-score, ROC-AUC, FPR, and FNR. PR-AUC was also recorded where class imbalance made it informative. Results across seeds were reported as mean and standard deviation, and paired changes were calculated because the same seeds and test identities were used in the compared conditions.

Accuracy was not treated as sufficient for deployment-like tests. When legitimate URLs greatly outnumber phishing URLs, a small FPR can produce many warnings, while a high FNR represents missed attacks. Therefore, low-prevalence tests used both class-independent metrics and the resulting confusion matrix.

4.11.2 Evaluation levels

The following tests were kept separate:

- the 49,229-row original internal test;
- identity-disjoint external sets, including frozen 10K, 20K, and 30K scenarios;
- later low-prevalence tests containing approximately 3–10% phishing;
- long-legitimate-URL stress tests;
- PhishTank-focused phishing snapshots;
- frozen synthetic challenge sets not used to select augmentation; and
- browser ONNX parity, latency, duplicate-scan, and user tests.

The source and date of each external dataset were stored with its result. Test identities were checked against training, calibration, and synthetic data before metrics were calculated.

4.11.3 DNS secondary experiment

A separate experiment evaluated a real-data DNS classifier as secondary evidence for uncertain URLs. It compared real-only and GAN-augmented primary URL models while holding the DNS data and decision rule fixed. GANs generated URL text only and did not create DNS observations.

The DNS experiment is supporting research rather than a feature silently added to the browser model. The distributed extension uses the URL ensemble and its local policy; this distinction is maintained in the implementation and results chapters.

4.11.4 Browser user-evaluation measures

The browser exports do not contain independently verified labels for every visited URL. Formal accuracy, precision, recall, F1, ROC-AUC, FPR, and FNR therefore cannot be calculated from voluntary user feedback alone. Three descriptive measures are used instead.

Let N be unique evaluated URLs, R be explicit false-alarm reports, and A be caution, warning, or block decisions. The user-reported false-alarm rate is

$$\text{URFAR} = \frac{R}{N} \times 100. \tag{4.4}$$

The overall alert rate is

$$\text{OAR} = \frac{A}{N} \times 100, \tag{4.5}$$

URFAR is a lower-bound descriptive rate unless the participant confirms that all observed mistakes were reported. OAR measures warning burden and is not called an FPR because some alerts may be correct. Feedback completeness is recorded from direct participant confirmation; it is not estimated by dividing reports by alerts, because feedback can be submitted from several decision tiers. Labelled frozen datasets remain the source of formal classifier metrics.

4.12 Browser System Design and Privacy

The browser extension performs inference locally. The URL is canonicalized and encoded using preprocessing resources packaged with the extension. Three ONNX models produce scores, which are averaged and mapped to allow, secondary review, caution, warning, or block tiers. Passive mode reports the decision without navigation blocking; protection mode can display stronger warnings at the frozen policy thresholds. Literal local and public IP addresses follow a separate conservative rule because a domain-name model lacks normal hostname context for them.

Repeated browser events can fire for the same navigation, history change, or fragment. The user-evaluation build therefore deduplicates canonical URL assessments and reports model executions separately from unique URL decisions. This prevents repeated scans from inflating the study denominator.

Participation requires explicit local consent. Automatic raw URLs are not uploaded or exported. Salted local fingerprints support deduplication, but the fingerprints themselves are not included in the exported result. A URL is stored in a feedback record only after the user deliberately reports a false alarm or missed phishing case, and query values and fragments are removed from that report. These controls reduce privacy risk while still permitting a small usability evaluation.

4.13 Reproducibility Controls

The following controls were used throughout the final experiments:

- configuration files store model dimensions, learning rates, epoch limits, seeds, augmentation ratios, and generator schedules;
- dataset and model files are identified by cryptographic hashes;
- training histories store each executed epoch and the retained epoch;
- checkpoints, predictions, thresholds, confusion matrices, and metric summaries are saved under versioned artifact directories;
- synthetic selection rules and accepted pools are frozen before final evaluation;
- internal, external, calibration, challenge, and browser results are stored separately; and
- earlier project versions remain archived instead of being overwritten by the final configuration.

These measures allow the final model to be reconstructed and prevent a good result from depending on an undocumented dataset or threshold change.

4.14 Chapter Summary

The methodology uses a fixed-label URL classification problem, hostname-based splits, explicit source roles, controlled real-only and GAN-augmented training, three-seed evaluation, and validation-selected checkpoints and thresholds. The final system combines a character CNN with 35 lexical features and uses a three-seed ensemble. MS-HardGAN-C generates source- and pattern-conditioned phishing URLs that must pass realism, novelty, diversity, and overlap checks before augmentation.

Internal, external, low-prevalence, synthetic-challenge, DNS, and browser tests are kept distinct. Formal classifier metrics are calculated only when verified labels are available; browser feedback instead uses the reported-false-alarm rate, overall alert rate, and participant-confirmed completeness status. The next chapter describes the implemented system at a component level.

Chapter 5

System Implementation

This chapter presents the practical implementation of the proposed phishing URL detection system. It explains how the data-processing pipeline, classifiers, GAN augmentation, hybrid model, and browser extension were joined into one working system. Detailed source-code descriptions are omitted because the purpose of this chapter is to describe the implemented system rather than individual programming instructions.

5.1 Implementation Overview

The project was implemented mainly in Python. PyTorch was used for the neural networks and GANs, while scikit-learn was used for the Random Forest (RF) and Support Vector Machine (SVM) baselines. Neural training was performed using an NVIDIA RTX 3060 Laptop GPU through CUDA. The browser extension was implemented in JavaScript using the Chromium Manifest V3 standard, and the selected model was converted to ONNX for local browser inference.

The implementation followed the experimental method established in Chapter 4. Data preparation, model training, GAN generation, evaluation, and browser deployment were kept as separate stages. This separation made it possible to repeat an experiment without changing the frozen training and evaluation data.

5.2 Project Structure

The earlier experiments were preserved as Version 1 and Version 2. Version 3 contains the final organised implementation. Its main folders separate source code, prepared data, trained models, experimental results, graphs, and the browser extension. This organisation prevents an old checkpoint or dataset from being confused with the final experimental materials.

Table 5.1: Main technologies used in the implemented system.

Part	Technology	Main purpose
Data processing	Python, SQLite, NumPy	Cleaning, splitting, encoding, and storing URLs
Traditional classifiers	scikit-learn	RF and SVM training and evaluation
Neural models and GANs	PyTorch and CUDA	CNN, RNN, LSTM, hybrid, and GAN training
Result production	JSON, CSV, Matplotlib	Saving metrics and producing graphs
Browser deployment	JavaScript and ONNX Runtime Web	Local URL classification in the browser
Extension platform	Chromium Manifest V3	Navigation monitoring, warnings, and local feedback

The main experimental settings are stored in a central configuration. These settings include the data split, URL length, model dimensions, batch sizes, training limits, random seeds, and GAN augmentation ratios. Therefore, the important settings remain consistent across repeated runs. Each completed run also records the model, seed, dataset identity, selected threshold, and final metrics.

5.3 Data Preparation

The selected multi-source dataset was processed into one controlled dataset for classifier training and evaluation. Each input URL was assigned a binary label: legitimate or phishing. Invalid labels, malformed entries, duplicate URLs, and conflicting records were removed or placed in a quarantine record instead of being silently accepted.

URLs were normalised before splitting. This included removing unnecessary spaces, standardising the hostname, handling the scheme consistently, and checking the port and URL structure. The normalised hostname was then used to create deterministic training, validation, and test partitions. Keeping the same hostname in one partition reduces identity leakage and provides a more meaningful internal evaluation.

The character vocabulary was built from the permitted training data. Every URL was converted into a fixed-length sequence of 128 character identifiers. The prepared arrays were stored in a form that could be read efficiently during training. Human-readable manifests and dataset hashes were also saved so that the origin and identity of the rows could be checked later.

5.4 Feature Extraction

Two representations of a URL were used in the project. The first was the raw character sequence, which allowed neural networks to learn local combinations of letters, digits, and symbols. The second was a set of 35 lexical and structural measurements used by the final hybrid model.

The lexical features describe properties such as URL length, hostname length, path and query length, number of dots and subdomains, digits, hyphens, special characters, entropy, suspicious terms, IP-address use, and domain-suffix structure. Exact domain identities were not used as input features. Numerical features were scaled using values learned only from the training partition, and the same stored values were later used during testing and browser inference.

This combination was selected because the character representation can learn patterns directly from URL text, while the lexical representation provides explicit structural information. The complete feature definitions and their research purpose were presented in Chapter 4.

5.5 Classifier Implementation

The implemented baselines were RF, SVM, CNN, RNN, and LSTM. RF and SVM used the lexical representation, whereas the neural baselines used the encoded character sequence. These models provided different comparison families rather than several minor variations of the same algorithm.

The final hybrid classifier contains two branches. The first is a character-level CNN that learns patterns directly from the URL string. The second is a small neural branch that processes the 35 lexical features. Their learned representations are combined before the final phishing probability is produced. This design permits the classifier to use both automatically learned text patterns and explicit URL structure.

Table 5.2: Implemented classifiers and their input representations.

Classifier	Input	Role in the study
RF	Lexical features	Non-linear traditional baseline
SVM	Lexical features	Linear margin-based baseline
CNN	URL characters	Learns short local character patterns
RNN	URL characters	Recurrent sequence baseline
LSTM	URL characters	Recurrent model with longer memory
Hybrid	Characters and lexical features	Final combined classifier and deployment candidate

All models followed the same experimental partitions and registered random seeds. Neural models used class-aware loss weighting, validation-based early stopping, and

GPU training. The best validation checkpoint was retained rather than automatically using the last epoch. Decision thresholds were selected from validation data and then frozen before testing. The test data were used only for the final measurement of a completed condition.

5.6 GAN Implementation and Augmentation

The project implemented multiple URL-generating GAN approaches during its development, including SeqGAN, RelGAN, URL-GAN, and the final multi-source hard-example GAN configuration (MS-HardGAN-C). These generators were trained using phishing URLs from the training partition only. Their purpose was to learn phishing-like character patterns and produce additional candidate URLs; they were not treated as systems capable of predicting future attacks.

Training began with a supervised sequence-learning stage so that a generator could first learn valid URL structure. Adversarial training was then introduced: the discriminator learned to distinguish real phishing URLs from generated ones, while the generator learned to produce more realistic outputs. The final hard-example configuration additionally used frozen classifiers to identify generated samples that were difficult for more than one classifier family.

Generated URLs were not added directly to the training set. They first passed validity, realism, uniqueness, and overlap checks. Obvious formatting errors, duplicates, copies of real training URLs, and unsuitable structural outliers were rejected. The accepted URLs formed frozen synthetic pools from which the registered augmentation percentages were created.

The real-only and augmented conditions retained the same real training rows. Their only intended difference was the addition of the selected synthetic phishing URLs. This controlled design allowed later comparisons to measure the effect of GAN augmentation rather than a change in the underlying real data.

For the direct SeqGAN–MS-HardGAN-C comparison, SeqGAN was retrained from scratch on the exact 84,280 phishing token rows used by MS-HardGAN-C. SeqGAN completed five supervised language-model epochs and three adversarial epochs, then produced 500,000 candidates. The standard filters accepted 344,507 URLs; the accepted pool had 100% uniqueness and validity and no exact accepted memorisation. After a second overlap check against the complete HYB-MS real training pool, exactly 40,410 URLs were frozen for the 10% condition. New HYB-MS models were trained for seeds 42, 43, and 44. The MS-HardGAN-C models used the matching real-row count, synthetic-row count, architecture, sampler, features, and threshold-selection rule. Separate output directories preserved the earlier models and results.

5.7 Training and Evaluation Process

Figure 5.1 summarises the complete implemented flow. The real data are prepared once and used for classifier training. The phishing training subset also supplies the GANs. Accepted generated URLs are combined with the same real training partition to form the augmentation conditions. Real-only and augmented models are then evaluated using unchanged internal and external datasets.

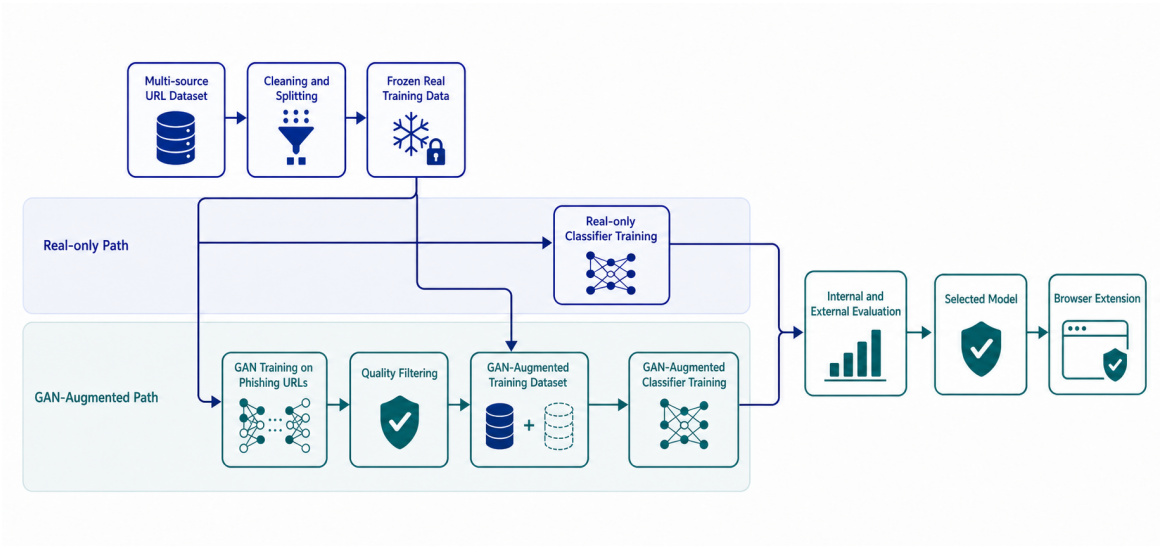


Figure 5.1: Parallel real-only and GAN-augmented training, evaluation, and deployment workflow.

For every evaluation, the system stored the confusion matrix and the required metrics: accuracy, precision, recall, F1-score, ROC-AUC, false-positive rate (FPR), and false-negative rate (FNR). PR-AUC was also stored where relevant. Results across seeds were summarised using the mean and standard deviation. External evaluation reused the frozen model and threshold without retraining.

Model checkpoints, experiment settings, predictions, metrics, and training histories were saved for traceability. Tables and graphs were produced from these stored files rather than copied manually from terminal output. The full experimental results are presented in the next chapter.

5.8 Browser Extension Implementation

The selected three-seed hybrid ensemble was converted from PyTorch to ONNX and included in the browser extension. The extension performs inference locally, so ordinary URL classification does not require a remote prediction server. The Python and JavaScript preprocessing outputs were compared before the model bundle was accepted, ensuring that the browser receives the same character and lexical

representations used during evaluation.

When a user navigates to a page, the extension normalises the URL, calculates its features, obtains the ensemble score, and applies the selected warning policy. The interface supports non-blocking caution messages, a secondary review state, and an interrupting warning for the highest-risk decisions. A passive demonstration mode reports the decision without redirecting the user. Trusted sites and IP-address inputs are handled separately to reduce avoidable false alarms.

Repeated browser events can otherwise cause one page to be scanned many times. The extension therefore prevents simultaneous duplicate scans and reuses a recent result for the same tab and URL. For user evaluation, a canonical URL is counted once as a unique observation, while repeated events are recorded only as technical reliability information.

5.9 Local Feedback and Privacy

A separate user-evaluation build was created without replacing the stable extension. Participants must agree before local study counters begin. The extension does not automatically upload browsing history. Instead, results are stored on the participant's device and can be exported voluntarily after the evaluation.

Users can report a false positive when a legitimate page receives an incorrect phishing warning, or a false negative when a phishing page is not warned about appropriately. A URL is stored only when the participant explicitly submits a feedback report. Query values and fragments are removed before export to reduce the chance of including personal information. The exported summary contains decision counts, warnings, errors, timing information, and the submitted feedback required for the evaluation.

5.10 Verification and Reproducibility

Automated checks were used for dataset integrity, preprocessing, feature dimensions, model inputs, metric calculations, browser policy, duplicate-scan handling, and agreement between PyTorch and ONNX predictions. Important data and model files were identified by hashes, and experiment outputs were stored under separate condition and seed directories.

These controls do not guarantee that every future URL distribution will match the research data. They do, however, ensure that the reported experiments can be traced to specific datasets, settings, checkpoints, and thresholds. This is necessary for a fair comparison between real-only and GAN-augmented models.

5.11 Chapter Summary

The proposed system was implemented as an organised pipeline connecting data preparation, baseline classifiers, a hybrid classifier, GAN-based URL augmentation, controlled evaluation, and browser deployment. The final hybrid model combines character-level CNN learning with lexical URL features, while the GAN stage supplies filtered synthetic phishing examples for controlled augmentation experiments.

The selected ensemble was exported to ONNX and integrated into a Manifest V3 extension that performs local inference, presents tiered warnings, reduces duplicate scans, and supports consent-based local feedback. The following chapter reports the internal, augmentation, external, and threshold results produced by this implementation. Browser operation and user evaluation are then presented separately in Chapter 7.

Chapter 6

Experiments and Results

6.1 Introduction

This chapter presents the experimental evidence used to answer the research questions defined in Chapter 1. The results are organised in five parts: classifier baselines, generated-URL quality, controlled GAN augmentation, external generalisation, and threshold selection. Browser implementation and user evaluation are reserved for Chapter 7.

The primary comparisons use fixed real training identities, unchanged real validation and test sets, and seeds 42, 43, and 44. Synthetic URLs are added only to the training partition. Unless otherwise stated, percentage results are the mean across the three seeds. Accuracy is reported for completeness, but it is not interpreted alone because phishing prevalence varies substantially between evaluation sets. Recall, precision, F1-score, ROC-AUC, FPR, and FNR are considered together.

6.2 Development Results from Versions 1 and 2

Versions 1 and 2 established the first working baselines and exposed the main experimental risk: high performance on a familiar random test split did not guarantee equal performance on URLs collected from another source. Their results are therefore retained as development evidence, but are not combined statistically with Version 3 because the preprocessing, stored splits, and evaluation protocols differ.

The Version 2 source-discrimination experiment obtained ROC-AUC 1.000 when separating internal and outside URLs within each class. This did not mean the phishing detector was perfect. It showed that source-specific patterns were so strong that a separate classifier could identify where a URL came from. This finding motivated the multi-source design, identity controls, and external evaluation used in Version 3.

Table 6.1: Summary of the first two experimental versions.

Version	Main real-data setting	Main internal result	Experimental lesson
V1	2,568,184-row source file; 252,430-row internal test	LSTM F1 97.17%; CNN F1 97.05%	A large random split produced strong internal results, but did not establish source-independent generalisation.
V2	Normalised frozen split: 2,055,821 train, 254,298 validation, 252,165 test	Three-seed mean F1: LSTM 96.81%; CNN 96.72%	Difficult generated samples improved synthetic-challenge recall, while real-test F1 generally remained strongest without augmentation.

6.3 Version 3 Real-Only Baselines

The five primary baseline families were trained on the same 358,619 permitted real URL identities and evaluated on the same 49,229-row internal test set. Their input representations differed only where required by the declared model family: RF and SVM used lexical features, the neural baselines used character sequences, and the later hybrid combined both branches.

Table 6.2: Version 3 real-only baseline results on the internal test set (mean of three seeds, %).

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	FPR	FNR
CNN	98.89	97.46	97.25	97.36	99.70	99.35	0.67	2.75
LSTM	98.98	98.17	96.96	97.56	99.68	99.31	0.48	3.04
RNN	98.66	97.30	96.29	96.79	99.39	98.79	0.71	3.71
RF	94.96	97.18	78.22	86.67	96.27	93.30	0.60	21.78
SVM	89.75	96.87	52.80	68.35	92.98	87.24	0.45	47.20

LSTM obtained the highest internal F1-score and accuracy, while CNN produced the highest recall and ROC-AUC. RF and especially SVM produced high precision but missed substantially more phishing URLs. This is visible in their FNRs of 21.78% and 47.20%. The result confirms that a high precision value cannot be used as the only basis for model selection.

These are internal results on a controlled split. They show that the models learned the registered task, but do not by themselves demonstrate deployment performance. The external experiments later in this chapter provide the more difficult generalisation test.

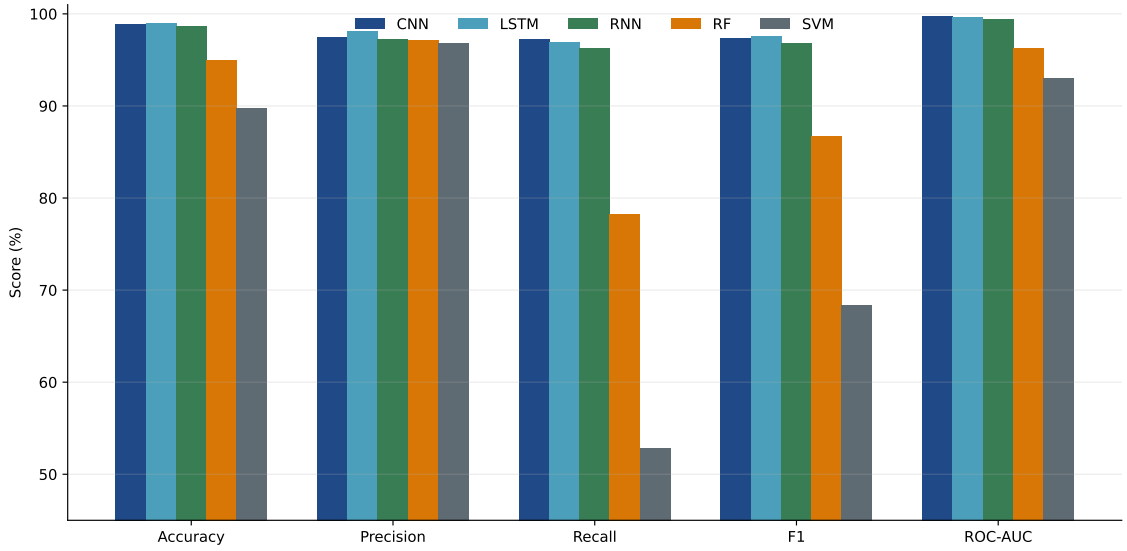


Figure 6.1: Comparison of the Version 3 real-only baseline classifiers.

6.4 Quality of Generated Phishing URLs

6.4.1 Initial generator comparison

Each initial GAN produced 500,000 candidate strings. Quality was measured before downstream classifier training. The main checks were uniqueness, parse-validity, strict acceptance, overlap with real data, lexical similarity, coverage of real-data clusters, and difficulty for frozen classifiers.

Table 6.3: Quality comparison of the initial Version 3 generators.

GAN	Unique	Parse-valid	Strictly accepted	Acceptance	Real-cluster coverage	Source AUC	Majority challenging
SeqGAN	99.839%	98.171%	349,687	69.94%	94%	0.7357	10.92%
URL-GAN	99.806%	98.154%	239,368	47.87%	90%	0.8817	5.06%
RelGAN	99.388%	81.428%	12,869	2.57%	86%	0.9888	0.085%

All three generators had high raw uniqueness and zero exact memorisation in the accepted comparison. However, RelGAN's strict acceptance was only 2.57%. Its source AUC of 0.9888 also showed that generated and real URLs were easily distinguished. This demonstrates why uniqueness alone is not sufficient: malformed or unrealistic strings can still be unique. SeqGAN gave the best combination of acceptance, real-cluster coverage, lexical closeness, and classifier challenge, and was therefore the strongest initial generator.

6.4.2 MS-HardGAN-C quality

MS-HardGAN-C was developed after the initial comparison to include source and structure conditions and a controlled classifier-challenge signal. From 630,000 raw candidates, 309,208 passed the complete acceptance procedure.

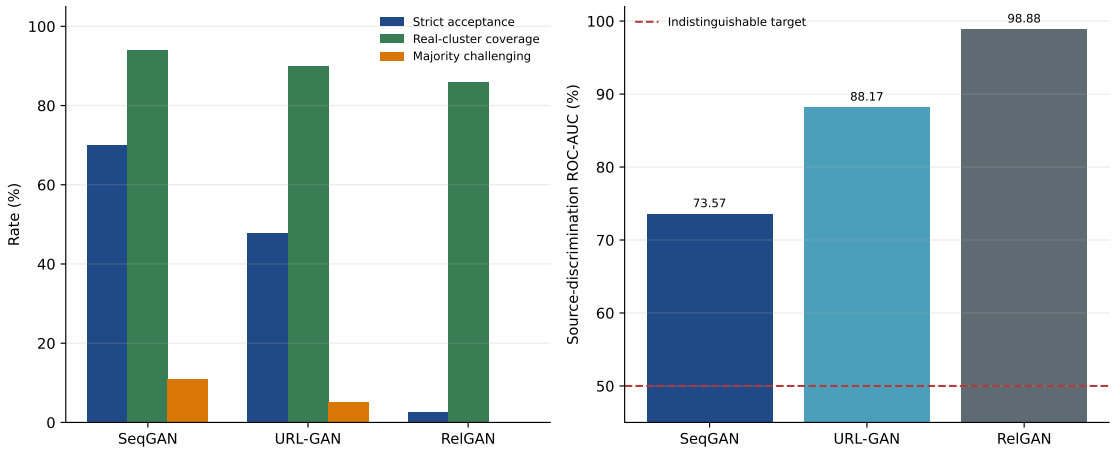


Figure 6.2: Generated-URL quality measures for SeqGAN, URL-GAN, and RelGAN.

Table 6.4: MS-HardGAN-C generation and filtering results.

Measure	Result	Interpretation
Raw uniqueness	99.93%	Candidate duplication was rare.
Raw URL-like validity	94.47%	Most candidates had a recognisable URL structure before strict filtering.
Strictly accepted	309,208 (49.08%)	About half passed every registered filter.
Accepted uniqueness and validity	100% and 100%	The final pool contained only unique accepted samples.
Real URL/hostname overlap	0	No accepted sample exactly matched permitted real URLs or hosts.
Real-cluster coverage	90.00%	The pool represented most registered real lexical clusters.
Source ROC-AUC	0.7992 ± 0.0055	Real and generated samples remained distinguishable, but not trivially so.
Majority challenging	9.95%	About one tenth challenged a majority of frozen classifier families.
At least one classifier challenging	52.83%	More than half challenged at least one family.

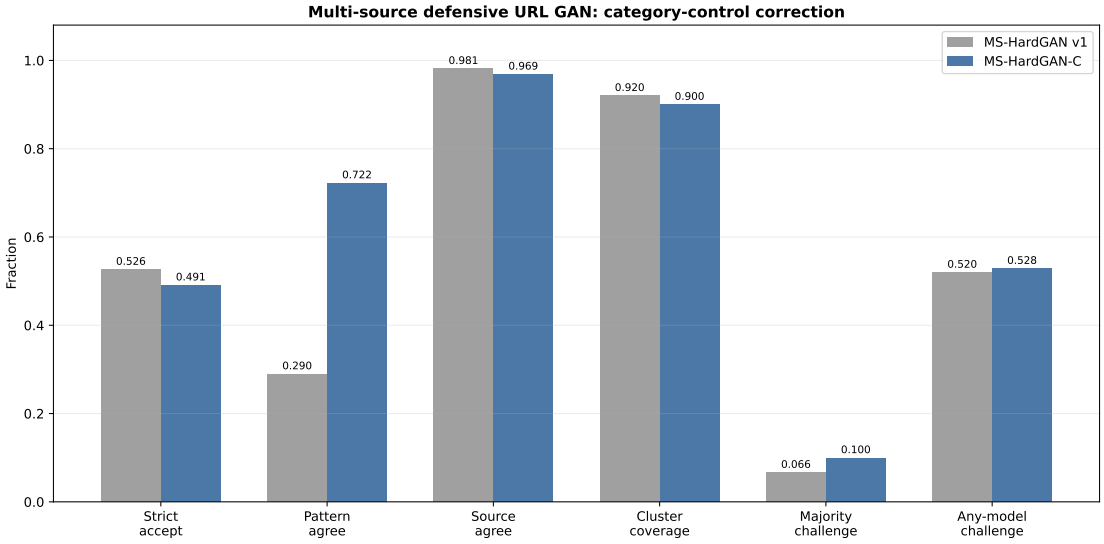


Figure 6.3: Quality profile of the accepted MS-HardGAN-C URL pool.

The quality tests support using this pool for augmentation experiments, not claiming that it represents unknown future attacks. The generator recombines patterns learned from known phishing data and can create difficult, novel stress examples. It cannot foresee an attack pattern for which there is no related evidence in the training distribution.

6.5 Controlled Augmentation Results

6.5.1 SeqGAN augmentation of the baselines

The first Version 3 augmentation experiment retained all 358,619 real training URLs and tested synthetic shares of 10%, 20%, 30%, and 40%. The table below reports F1-score because it summarises precision and recall; the complete multi-metric comparison is shown in Figure 6.4.

Table 6.5: Internal F1-score under SeqGAN augmentation (mean of three seeds).

Model	Real-only	10%	20%	30%	40%
CNN	0.9736	**0.9764**	0.9758	0.9751	0.9724
LSTM	**0.9756**	0.9722	0.9744	0.9750	0.9731
RNN	**0.9679**	0.9679	0.9532	0.9639	0.9623
RF	**0.8667**	0.8625	0.8576	0.8508	0.8433
SVM	0.6835	0.6924	**0.6935**	0.6920	0.6887

SeqGAN produced small, model-specific gains: CNN was strongest at 10% and SVM at 20%. The improvement did not hold across all classifiers. LSTM and RF were strongest without augmentation, and higher synthetic ratios generally reduced

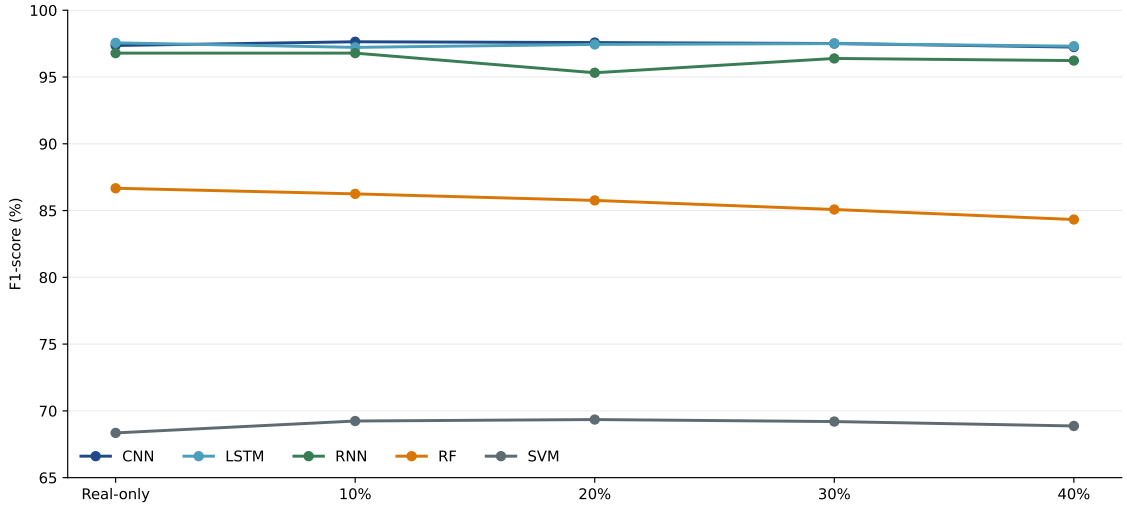


Figure 6.4: Internal multi-metric results for the SeqGAN augmentation ratios.

internal F1. The correct conclusion is therefore not that more synthetic data is always better, but that its effect depends on the model and synthetic share.

6.5.2 MS-HardGAN-C augmentation

The multi-source experiment used 363,692 fixed real URLs and nested synthetic pools of 40,410, 90,923, 155,868, and 242,461 URLs. On the unchanged real internal test set, real-only training remained strongest overall. For example, CNN F1 decreased from 0.9737 to 0.9728 at 10%, then to 0.9655 at 40%.

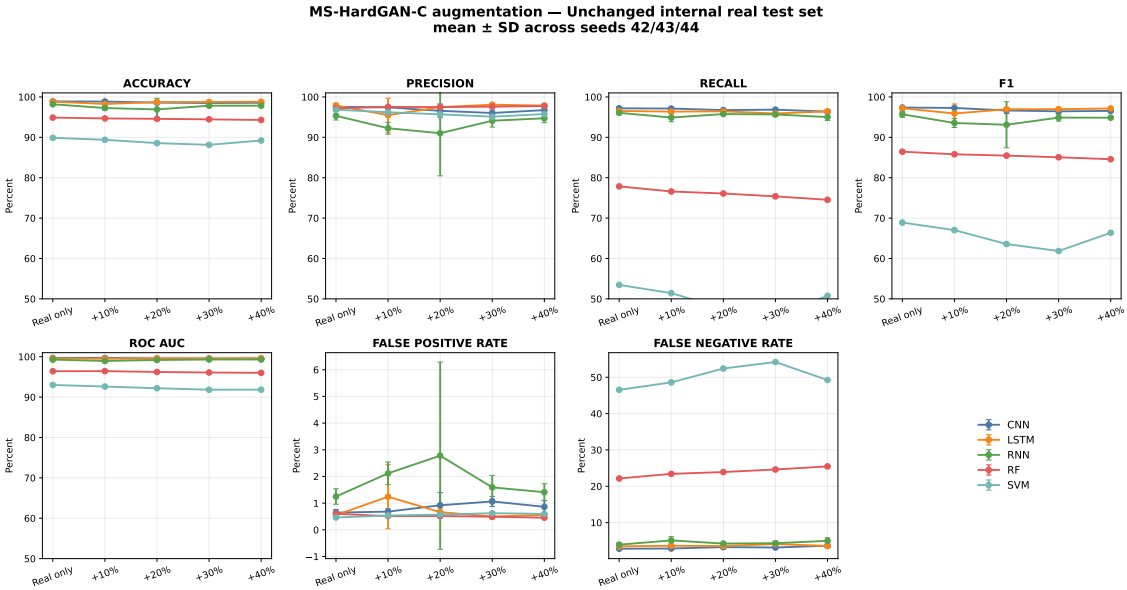


Figure 6.5: Internal real-test results under MS-HardGAN-C augmentation.

The separate 61,728-row synthetic challenge set gives a different view. It was held out from both training and synthetic selection. All model families gained challenge

recall at one or more ratios, as shown in Figure 6.6. CNN increased from 94.36% real-only recall to 98.20% at 10%, while LSTM increased from 90.57% to 97.38%. RF increased from 75.78% to 84.53%. These gains show stronger recognition of held-out generator-related patterns, but do not replace evaluation on real URLs.

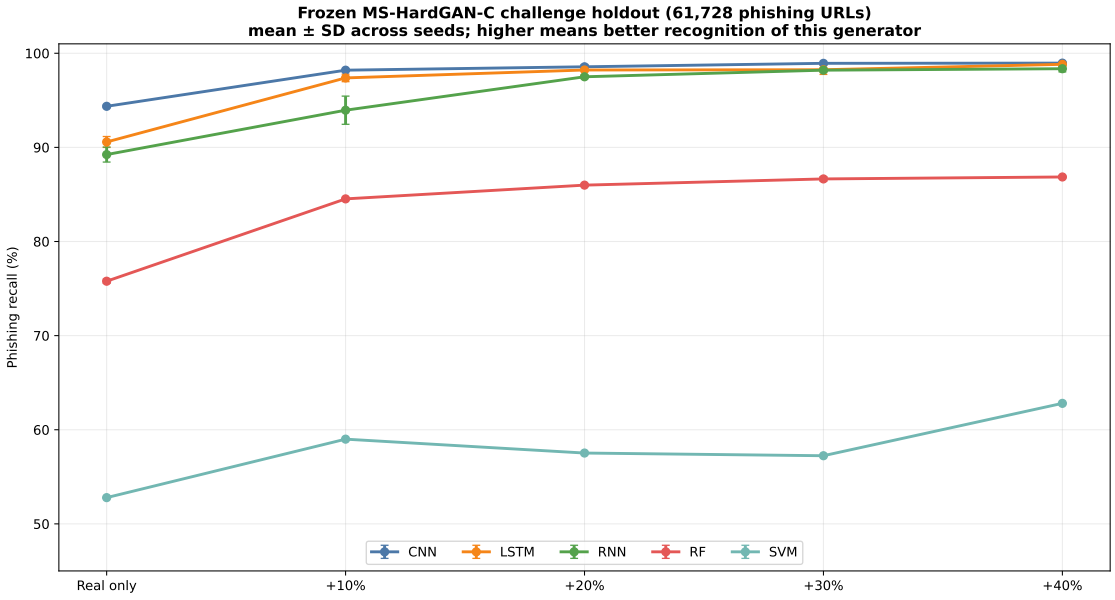


Figure 6.6: Recall on the frozen MS-HardGAN-C synthetic challenge set.

Taken together, the internal real and synthetic challenge results answer an important question. Lower real-test performance after augmentation does not mean that the classifier failed to learn the generated URLs. In this case the opposite occurred: synthetic-challenge recall increased while performance on the real distribution sometimes decreased. The evidence points to a distribution trade-off rather than simple failure to classify GAN samples.

6.5.3 Matched SeqGAN and MS-HardGAN-C comparison

The earlier SeqGAN and MS-HardGAN-C studies had different development roles and did not use exactly the same phishing input. To remove this input-data confound, SeqGAN was retrained on the exact 84,280 phishing rows used by MS-HardGAN-C. The matched SeqGAN generated 500,000 candidates, of which 344,507 passed strict filtering. Exactly 40,410 clean URLs were added to the same 363,692 real HYB-MS rows used by the MS-HardGAN-C 10% condition.

Both augmented conditions used the same HYB-MS architecture, weighted sampler, features, seeds 42/43/44, and frozen evaluation rows. Each three-seed ensemble selected its threshold using the same separate 400-row calibration cohort, maximising calibration F1 subject to calibration FPR no greater than 1%. Neither external set was used for threshold selection. The external 30K set in Table 6.6 contains 30.07%

phishing and is different from the later primary 30K low-prevalence test.

Table 6.6: Matched-input generator comparison using the same HYB-MS procedure (%).

Dataset	Condition	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	FPR	FNR
Internal	Real-only	96.08	99.95	81.35	89.70	99.64	99.23	0.01	18.65
	SeqGAN 10%	96.24	99.93	82.13	90.16	**99.72**	**99.35**	0.02	17.87
	MS-HardGAN-C 10%	**96.45**	99.93	**83.13**	**90.76**	99.55	99.09	0.02	**16.87**
External 10K	Real-only	91.77	98.04	85.24	91.20	96.63	97.53	1.70	14.76
	SeqGAN 10%	92.14	98.44	85.64	91.59	96.60	97.57	1.36	14.36
	MS-HardGAN-C 10%	**93.08**	**98.47**	**87.52**	**92.67**	**96.90**	**97.84**	1.36	**12.48**
External 30K	Real-only	94.64	96.54	85.25	90.54	96.95	96.05	1.32	14.75
	SeqGAN 10%	94.53	97.02	84.41	90.28	96.70	95.85	1.12	15.59
	MS-HardGAN-C 10%	**95.42**	**97.53**	**86.99**	**91.96**	**97.00**	**96.41**	**0.95**	**13.01**

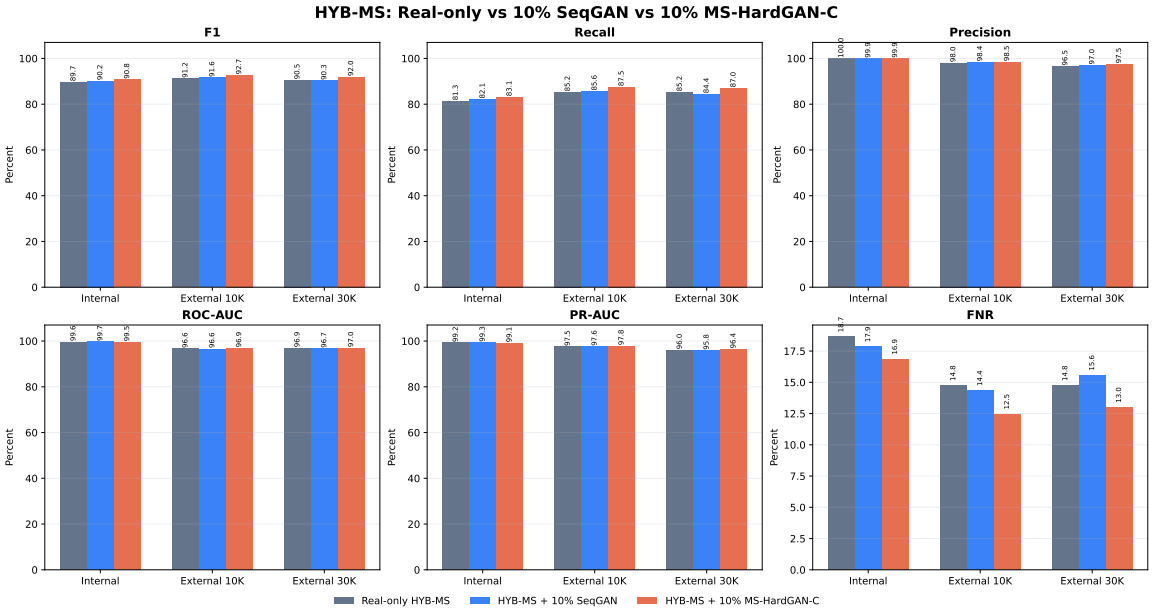


Figure 6.7: Real-only HYB-MS and the two matched-input 10% generator conditions.

Matching the phishing input substantially improved SeqGAN's downstream result. On the external 10K set, its F1 increased from 88.54% in the earlier unmatched comparison to 91.59%, and FNR fell from 20.14% to 14.36%. This confirms that generator training data was an important confound. MS-HardGAN-C nevertheless remained stronger after the confound was removed. At the same 1.36% external 10K FPR, it increased recall by 1.88 points and reduced FNR by 1.88 points relative to matched SeqGAN. On the external 30K set it also achieved higher F1, higher recall, lower FPR, and lower FNR. The result supports choosing MS-HardGAN-C for HYB-MS because its conditional structural, source, and classifier-challenge objectives produced a more useful augmentation pool, not because it had access to additional real phishing identities.

6.5.4 Final hybrid ratio comparison

The final HYB-MS comparison used the same hybrid architecture for the real-only and GAN-augmented conditions. The real rows, seeds, test rows, and evaluation rules were fixed. A balanced 10,000-row external set was used to compare the augmentation ratios with separately calibrated ensemble thresholds.

Table 6.7: Calibrated HYB-MS results on the frozen balanced 10K external set (%).

Condition	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	FPR	FNR
Real-only	91.77	98.04	85.24	91.20	96.63	97.49	1.70	14.76
GAN 10%	**93.10**	98.43	**87.60**	**92.70**	**96.89**	**97.81**	1.40	**12.40**
GAN 20%	91.30	98.96	83.48	90.56	96.30	97.44	0.88	16.52
GAN 30%	88.39	99.23	77.38	86.95	95.85	96.91	0.60	22.62
GAN 40%	85.15	**99.30**	70.80	82.66	95.55	96.64	**0.50**	29.20

The 10% condition gave the best balanced result. Larger ratios reduced FPR and raised precision, but the cost was a sharp loss of phishing recall. Therefore, 10% was retained as the main augmentation condition; it was not selected by changing the tested ratios after the final results were observed.

6.6 External Evaluation

6.6.1 Why prevalence matters

A balanced test is useful for comparing classes, but browser traffic normally contains many more legitimate than phishing URLs. In a low-prevalence setting, even a small FPR can produce many false warnings, while accuracy can appear high simply because most rows are legitimate. Later external snapshots therefore used phishing shares of approximately 5% and 3%, and results are reported with confusion-derived FPR and FNR.

6.6.2 External comparison of classifier families

The hybrid was not chosen only because it was the most complex model. A final equal-calibration comparison applied the same calibration data, threshold rule, balanced 10K and 30K external sets, and eligibility checks to the competing model families. Table 6.8 reports the balanced 10K results. The ordinary multi-source baselines and real-only HYB-MS used the same 363,692 stored real URL identities. HYB-MS additionally used its declared weighted source-sampling and class-conditional alignment procedure; these are part of the model design rather than extra unique training URLs.

Table 6.8: Equal-calibration external comparison on the balanced 10K set (%).

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	FPR	FNR
CNN-MS	55.47	95.51	11.48	20.50	79.83	81.12	**0.54**	88.52
LSTM-MS	90.63	98.54	82.48	89.80	93.72	96.08	1.22	17.52
RNN-MS	66.79	96.66	34.78	51.15	89.01	91.29	1.20	65.22
RF-MS	65.13	96.04	31.56	47.51	63.01	74.37	1.30	68.44
SVM-MS	52.95	86.78	6.96	12.89	24.99	43.72	1.06	93.04
HYB-MS real-only	91.77	98.04	85.24	91.20	96.63	97.49	1.70	14.76
HYB-MS +10%	**93.10**	98.43	**87.60**	**92.70**	**96.89**	**97.81**	1.40	**12.40**

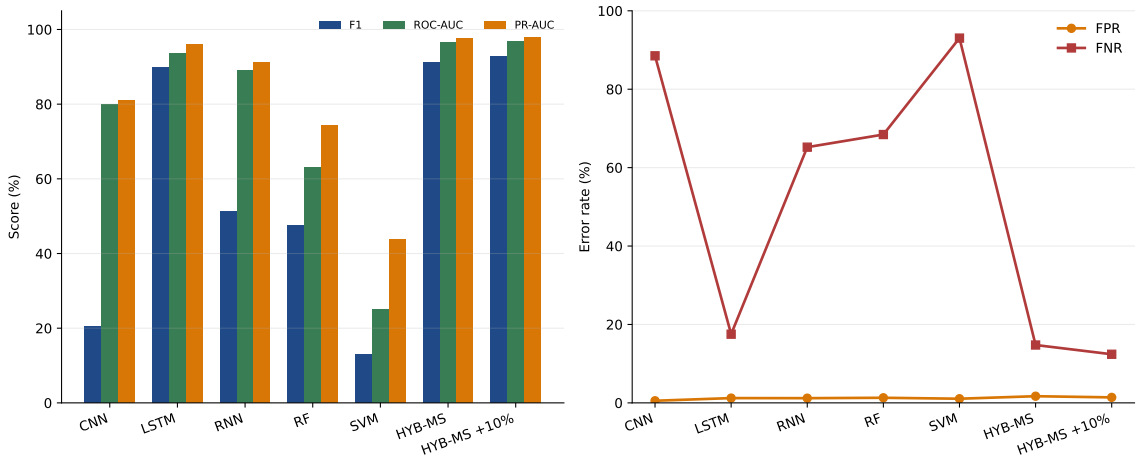


Figure 6.8: External performance of the equally calibrated classifier families.

LSTM-MS was the strongest non-hybrid external baseline, with F1 89.80%. Real-only HYB-MS improved F1 to 91.20%, ROC-AUC to 96.63%, and FNR to 14.76%. The 10% GAN-augmented hybrid then produced the strongest external F1 and the lowest FNR among the eligible high-performing models. Although CNN-MS had the lowest FPR, it detected only 11.48% of phishing URLs; that operating point was not a useful security balance.

The hybrid was consequently selected for three connected reasons. First, its character CNN learns local spelling and delimiter patterns, while the lexical branch retains full-URL structural evidence, including information beyond the 128-character sequence limit. Second, weighted source exposure and class-conditional alignment were introduced specifically to reduce the source shift identified in Version 2. Third, the equal-calibration results show that these design choices improved external F1 and reduced missed phishing relative to the strongest ordinary baseline. The GAN contribution was then measured fairly by comparing HYB-MS real-only and HYB-MS +10%, which have the same architecture and differ only in the registered synthetic training rows.

6.6.3 Primary 30K low-prevalence confirmation

The frozen 30,000-row evaluation contained 28,500 legitimate and 1,500 phishing URLs. The five ordinary multi-source baselines, HYB-MS real-only, and HYB-MS with 10% augmentation were evaluated on these same rows. All checkpoints and equal-calibration thresholds remained frozen; no model was retrained or tuned for this comparison.

Table 6.9: All classifier results on the 30K external set with 5% phishing prevalence.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	FPR	FNR
CNN-MS	95.31%	66.32%	12.60%	21.18%	79.95%	31.81%	**0.34%**	87.40%
LSTM-MS	97.48%	74.65%	75.20%	74.93%	92.61%	81.70%	1.34%	24.80%
RNN-MS	96.16%	68.64%	42.60%	52.57%	88.46%	58.53%	1.02%	57.40%
RF-MS	95.13%	52.70%	25.40%	34.28%	53.56%	30.71%	1.20%	74.60%
SVM-MS	94.33%	26.74%	7.67%	11.92%	28.11%	8.28%	1.11%	92.33%
HYB-MS real-only	97.85%	77.16%	81.07%	79.06%	96.58%	84.29%	1.26%	18.93%
HYB-MS +10%	**98.12%**	**80.95%**	**81.60%**	**81.27%**	**96.82%**	**87.27%**	1.01%	**18.40%**

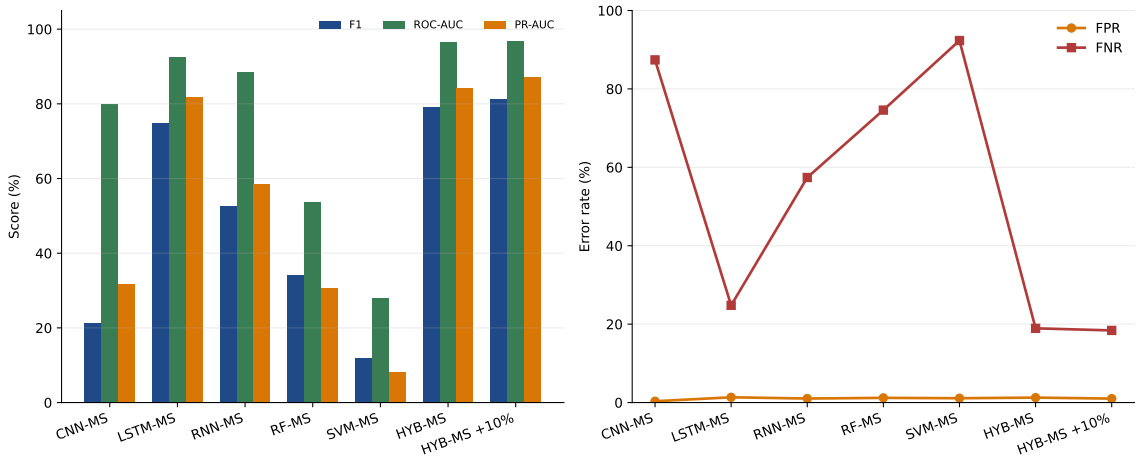


Figure 6.9: All classifiers on the frozen 30K low-prevalence external set.

LSTM-MS was again the strongest ordinary baseline, reaching 74.93% F1 and 75.20% recall. HYB-MS real-only improved these values to 79.06% F1 and 81.07% recall. HYB-MS +10% produced the strongest overall result: 81.27% F1, 81.60% recall, 96.82% ROC-AUC, and 87.27% PR-AUC. CNN-MS achieved the smallest FPR, but its 12.60% recall meant that it missed most phishing URLs; this was not an acceptable balance. The common dataset and frozen thresholds therefore provide direct external evidence for selecting the hybrid rather than selecting it only from its internal performance.

The 10% model produced 288 false positives and 276 false negatives, compared with 360 and 284 for real-only. McNemar's paired test over all changed decisions gave $p = 7.43 \times 10^{-5}$. The bootstrap 95% confidence interval for the F1 change was approximately +1.05 to +3.41 percentage points, while the recall change alone was not

significant on the phishing subset ($p = 0.526$). The most defensible statement is that 10% augmentation improved the overall decision balance and reduced false positives on this snapshot; evidence for a standalone recall improvement was weaker.

6.6.4 Independent PhishTank and later frozen snapshots

Additional snapshots tested whether the result repeated when the phishing source or prevalence changed. The PhishTank-only evaluation used verified PhishTank phishing URLs [16]. Legitimate rows were kept substantially more numerous than phishing rows in every low-prevalence test.

Table 6.10: External comparison of real-only and 10% GAN-augmented HYB-MS (%).

Dataset	Condition	Accuracy	Precision	Recall	F1	FPR	FNR
PhishTank 10K, 5%	Real-only	98.00	76.88	85.80	81.10	1.36	14.20
	GAN 10%	**98.45**	**81.89**	**88.60**	**85.11**	**1.03**	**11.40**
Final labelled 6K, 5%	Real-only	97.95	82.18	75.33	78.61	0.86	24.67
	GAN 10%	**98.23**	**84.15**	**79.67**	**81.85**	**0.79**	**20.33**
Post-freeze 10K, 3%	Real-only	98.71	77.14	81.00	79.02	0.74	19.00
	GAN 10%	**98.80**	**78.48**	**82.67**	**80.52**	**0.70**	**17.33**

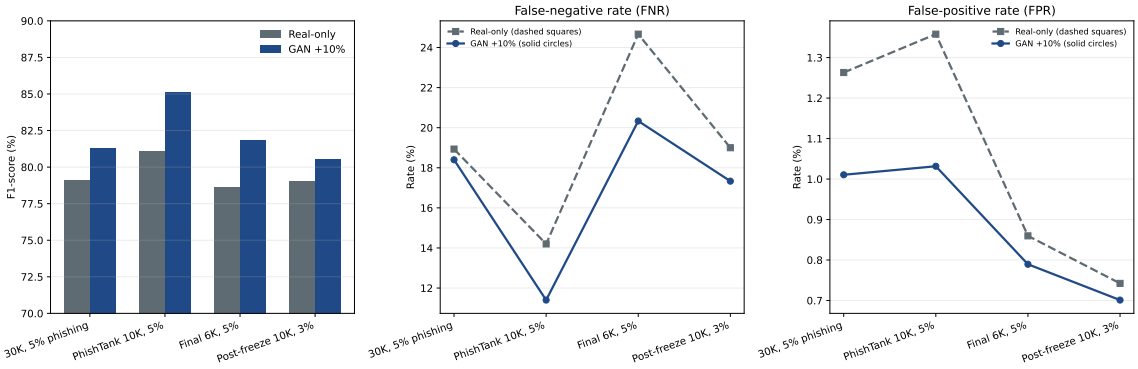


Figure 6.10: F1, FPR, and FNR across the low-prevalence external evaluations. Grey dashed lines with square markers represent the real-only model; blue solid lines with circular markers represent the 10% GAN-augmented model.

The 10% condition improved F1 and reduced both FPR and FNR in all four listed low-prevalence snapshots, including the 30K result in Table 6.9. The size of the gain varied, so it should be reported as repeated empirical improvement under these test conditions rather than proof that augmentation will improve every future dataset.

6.6.5 Long-URL stress test

Long legitimate URLs were deliberately included in a separate stress test because earlier browser observations showed that length could contribute to false warnings.

This set was a targeted diagnostic and not a representative estimate of ordinary traffic.

Table 6.11: Results on the long-URL stress set (%).

Condition	Accuracy	Precision	Recall	F1	ROC-AUC	FPR	FNR
Real-only	96.95	88.00	77.00	82.13	98.51	1.05	23.00
GAN 10%	96.86	84.66	80.00	82.26	98.42	1.45	20.00

GAN augmentation detected six additional phishing URLs but introduced eight additional false positives. F1 changed only slightly and the predeclared directional non-regression check was not passed. Consequently, long URLs remain a stated limitation. They should not be treated as phishing solely because of length; the full combination of character and lexical evidence is required.

6.7 Threshold Analysis

The model produces a continuous score, while a threshold converts the score to a class decision. Lower thresholds classify more URLs as phishing. This reduces FNR but usually increases FPR and can reduce precision, especially when real phishing prevalence is low. Thresholds were selected using calibration data, not the final test labels.

Table 6.12: Single-stage threshold policies on the registered threshold evaluation set.

Policy	Threshold	Precision	Recall	F1	FPR	FNR
Ultra-strict	0.999973	99.37%	31.60%	47.95%	0.02%	68.40%
Strict	0.999849	98.60%	45.75%	62.50%	0.07%	54.25%
Conservative	0.995319	94.83%	79.85%	86.70%	0.48%	20.15%
Balanced	0.970766	89.77%	87.35%	88.55%	1.11%	12.65%
Recall-first	0.885636	81.57%	90.75%	85.92%	2.28%	9.25%
High-recall	0.670915	64.84%	93.60%	76.61%	5.64%	6.40%

The high-recall threshold reduced FNR to 6.40%, but raised FPR to 5.64%. In a stream containing thousands of legitimate pages, this would create too many warnings. Conversely, the strict thresholds achieved very low FPR but missed more than half of phishing URLs. The balanced region provides a more practical starting point, while a browser can use score bands so that moderate evidence creates a caution rather than an immediate block. The implementation of those warning modes is discussed in Chapter 7.

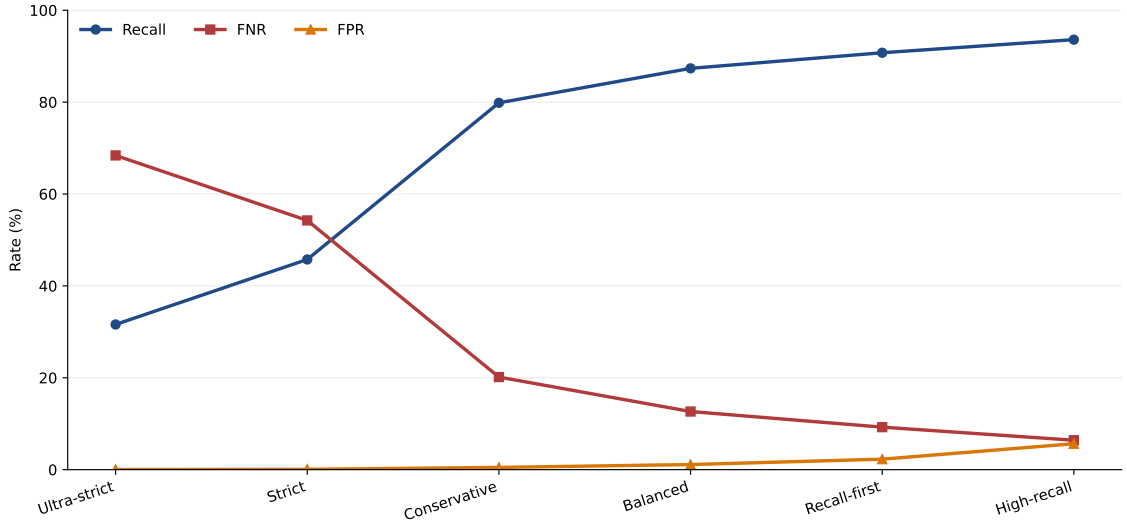


Figure 6.11: Recall, FPR, and FNR trade-off across the registered score policies.

6.8 Discussion of the Research Questions

6.8.1 Can a URL-only classifier detect phishing effectively?

Yes, within the evaluated distributions. CNN and LSTM achieved internal F1 above 97%, and the hybrid models produced useful discrimination on multiple external sets. However, external FNR was consistently higher than internal FNR. URL-only detection is therefore useful as a defensive signal, not a guarantee that every phishing page will be identified.

6.8.2 Does GAN augmentation improve detection?

This research question was investigated experimentally; it did not begin with the assumption that augmentation must improve every metric or every model. Random and high-ratio augmentation produced mixed results and sometimes reduced performance on the internal real test set. This negative evidence is part of the answer because it shows that the quantity and quality of generated URLs matter.

The controlled 10% MS-HardGAN-C condition gave the clearest positive result. Using the same hybrid architecture, real identities, seeds, calibration method, and test rows, it improved F1 and the confusion-matrix balance on several frozen low-prevalence external datasets. On the primary 30K set it also outperformed all five ordinary classifier families. The study therefore finds that carefully filtered GAN augmentation *can* improve URL-phishing detection under the evaluated conditions. This directly answers the research question while recognising that the observed benefit depends on the generator, augmentation ratio, classifier, and evaluation distribution.

The matched-input generator ablation strengthens this interpretation. After Seq-

GAN and MS-HardGAN-C were trained on the exact same 84,280 phishing rows, both 10% pools improved HYB-MS on the balanced external 10K set, but MS-HardGAN-C retained the higher F1 and lower FNR. Thus, its advantage cannot be explained only by its earlier access to 2,061 additional multi-source phishing rows.

6.8.3 Are the generated URLs useful beyond final accuracy?

Yes. The generators produced large novel pools with no exact accepted overlap with the permitted real URLs. Held-out synthetic-challenge recall increased substantially after augmentation, showing that the added samples expanded coverage of difficult generator-related patterns. The realism and source-AUC tests also identified poor generators that raw uniqueness would have accepted. These quality results are an independent contribution even where downstream real-test metrics did not improve.

6.8.4 What remains difficult?

Distribution shift remains the main limitation. Source-specific structure, long legitimate URLs, new phishing campaigns, and changing class prevalence all alter the error balance. Threshold lowering can reduce FNR, but it cannot eliminate missed phishing without increasing false warnings. Continued testing on later data is therefore necessary for any operational use.

6.9 Chapter Summary

The experiments established strong internal neural baselines, compared four implemented GAN designs using quality measures beyond uniqueness, and tested augmentation without forcing a positive outcome. SeqGAN was the strongest initial generator. When SeqGAN was later retrained on the same 84,280 phishing rows as MS-HardGAN-C, its HYB-MS augmentation result improved, but MS-HardGAN-C still supplied the stronger 10% pool. High augmentation ratios were generally harmful to real-test balance, whereas the 10% hybrid condition produced the most consistent external gains.

The results also showed that metric interpretation depends on prevalence and threshold choice. A reduction in FNR can be purchased at the cost of many false positives, and high accuracy can conceal weak phishing recall. Chapter 7 now examines how these findings were translated into browser warning behaviour and how the extension performed during technical and user evaluation.

Chapter 7

Browser Extension and User Testing

7.1 Introduction

This chapter describes how the selected HYB-MS+10% model was used in a working browser extension and how the browser-specific decisions were tested. The classifier score, the extension policy, and user feedback are reported as separate forms of evidence. This separation is important because a browser can change how a score is presented without changing the underlying model.

7.2 Local Detection Process

The extension is implemented for Chromium-based browsers using Manifest V3. The three frozen hybrid checkpoints were converted to ONNX and packaged with the same character vocabulary, maximum length, lexical-feature definitions, and scaling values used in Python. For each HTTP(S) navigation, the extension normalises the URL, forms the character and 35-feature inputs, obtains the three scores, and uses their mean as the ensemble score. Inference is performed on the user's device.

The browser policy then converts the score into an action. Secondary-review and caution results use non-blocking notices. High-risk and very-high-risk results pause navigation in Normal mode, whereas Warnings-only mode never pauses the page. A user can continue once or permanently allow an exact hostname; neither action changes the raw model score. Redirect destinations and single-page application history changes are evaluated as new destinations.

Table 7.1: Browser decision tiers and their user-facing behaviour.

Tier	Normal-mode action	Purpose
Allow	No notice	No strong URL-text warning was found.
Secondary review	Passive notice	An unusual URL can be reviewed without interruption.
Caution	Passive notice	Suspicious evidence is shown while the page remains available.
Warning	Navigation pause	A high-risk result is blocked, with a clear user override.
Block	Navigation pause	A very-high-risk result is blocked; the user may return, continue once, or allow the exact hostname.

7.3 Explainability and User Control

The extension provides short diagnostic reasons derived from the same lexical inputs used by the hybrid model, such as unusual length, many symbols, deep paths, encoded characters, explicit ports, IP-address hosts, and suspicious terms. These messages are debugging clues rather than causal explanations or proof of phishing. They are displayed inside the extension popup and on the full warning page. The smaller notice placed over an ordinary webpage shows only the result, exact hostname, and risk score, with a prompt to open the extension for details. This reduces interruption and keeps the detailed reason available when the user requests it.

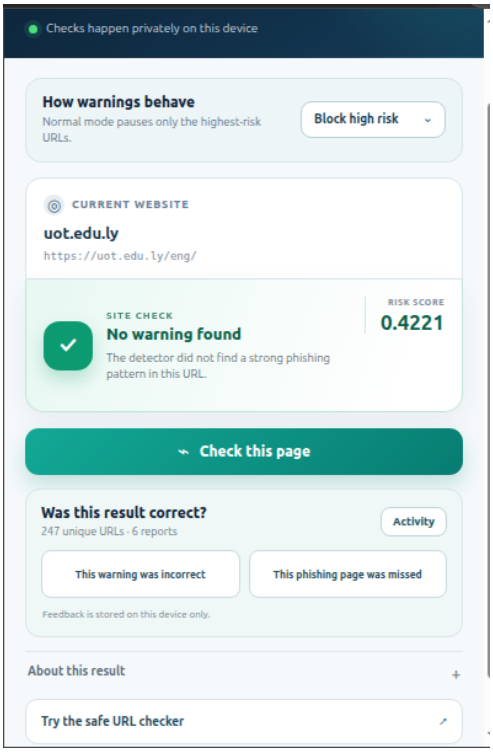
The permanent allow action now names the exact hostname, for example "Always allow `docs.google.com`", rather than using an ambiguous phrase such as "allow this site". Hostnames remain left-to-right in both the English and Arabic interfaces. Allowing a hostname suppresses later browser interventions for that exact name only; it does not include look-alike names or other subdomains.

7.3.1 Interface examples

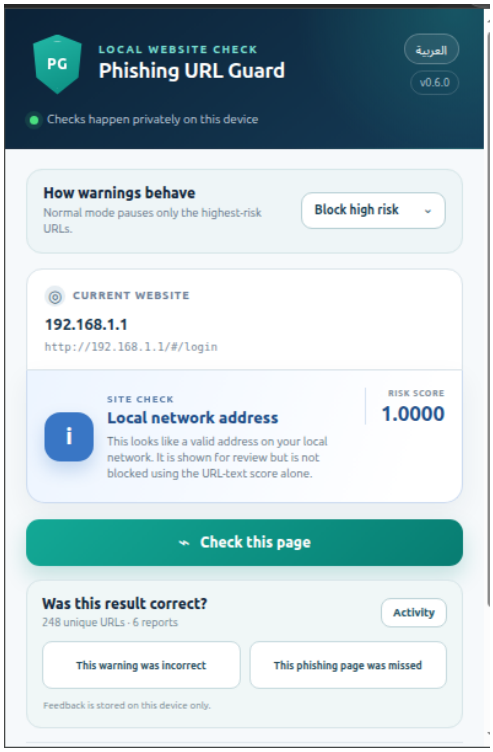
Figure 7.1 shows the compact interface used during ordinary browsing. The first view presents a legitimate university page with no warning. The second demonstrates the separate local-IP rule: the raw model score remains visible, but a valid private-network address is not blocked from URL text alone.

Figure 7.2 shows the full interruption page used for a very-high-risk result in Normal mode. It displays the destination and risk tier and offers three explicit choices: return to safety, continue once, or allow only the exact hostname.

Figure 7.3 illustrates the same type of model result in Warnings-only mode. The destination remains available and a compact notice is displayed instead of redirecting the tab.



(a) Ordinary extension result



(b) Private-network address handling

Figure 7.1: Extension popup examples for a normal website and a local IP address.

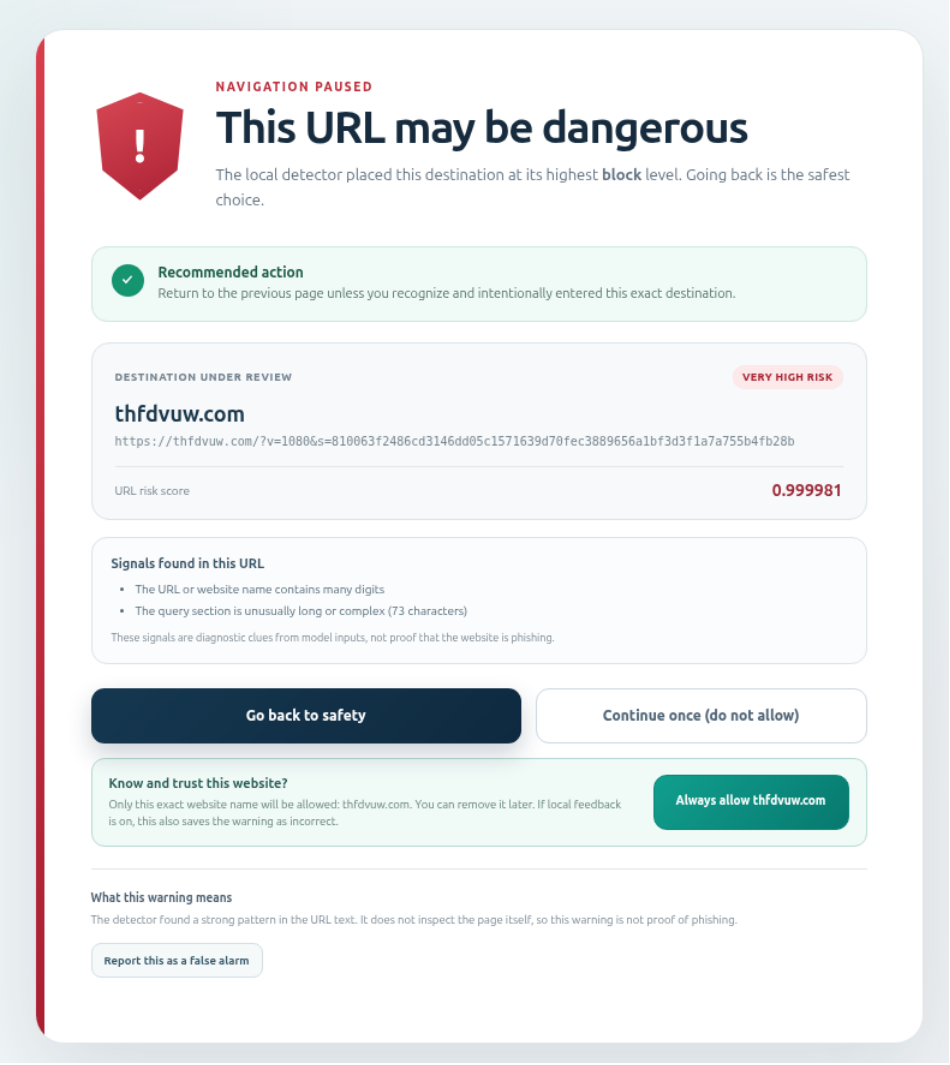


Figure 7.2: Full warning page shown when Normal mode pauses a very-high-risk URL.

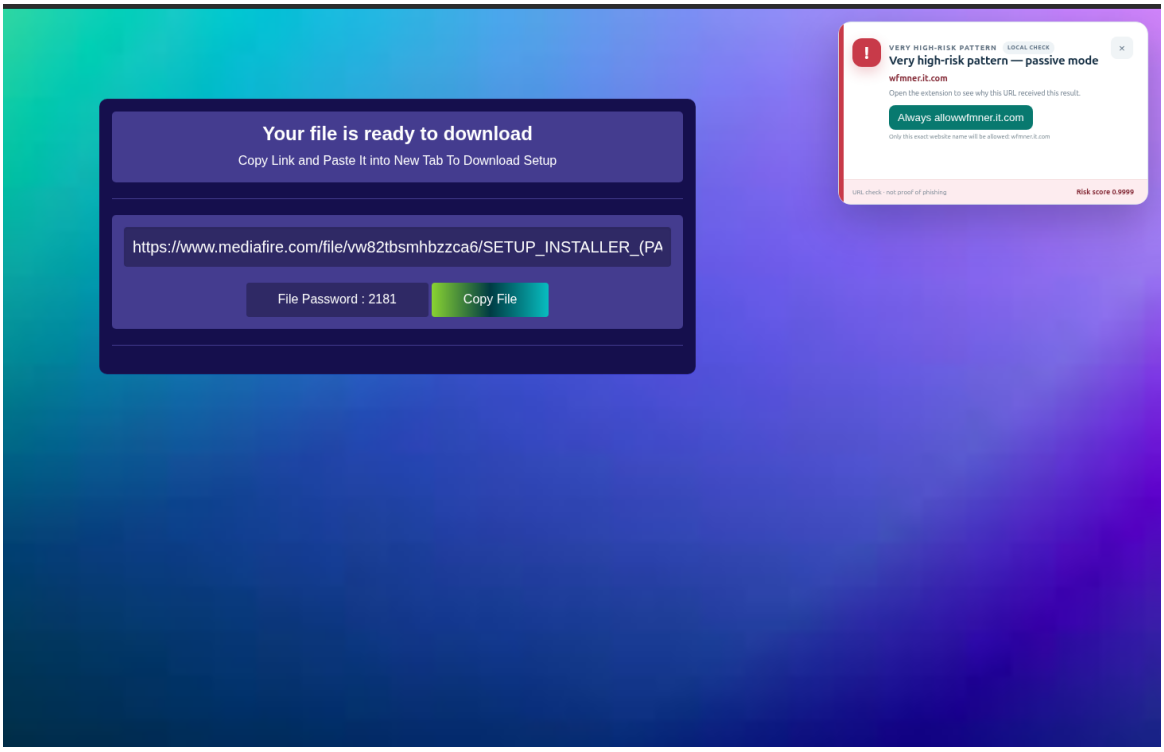


Figure 7.3: Non-blocking notice for a very-high-risk result in Warnings-only mode.

7.4 Reviewed Services and User-Controlled Content

A versioned catalogue reduces repeated false warnings on reviewed first-party service pages. Popularity alone does not grant trust. A rule requires HTTPS, an exact hostname, a standard port, no embedded credentials, and an allowed path. Outbound redirect routes and other excluded paths retain normal model handling.

ChatGPT is included through exact rules for `chatgpt.com`, `www.chatgpt.com`, and the legacy `chat.openai.com` hostname. The home page, conversation routes under `/c/`, authentication, settings, login, and sign-up routes are treated as fixed service pages. Public `/share/` pages and custom `/g/` routes remain model-checked because their content can be created or configured by users. Automated policy tests confirmed that normal ChatGPT conversation URLs are allowed quietly under the current catalogue.

7.5 Reliability and Privacy Controls

Browser navigation can generate repeated events for the same URL. The extension serialises overlapping checks, reuses a recent result for the same tab and URL, and keeps a salted local fingerprint so a repeated assessment is not counted as a new unique URL. Model executions, unique URL decisions, advisory displays, and actual navigation interruptions are stored as separate counters.

User-evaluation recording is optional and begins only after consent. Ordinary visited URLs are not exported as readable text. A readable URL is retained only after an explicit false-alarm or missed-phishing report, and its query values and fragment are removed. Nothing is uploaded automatically; users export their local results voluntarily.

7.6 User Evaluation

7.6.1 Method and browsing context

The user evaluation was designed to test the extension during ordinary browsing and to identify usability problems that are difficult to reproduce with a fixed dataset. Ten current-format exports were collected from Chrome, Chromium, Opera GX, Brave, and Edge. Four participants also deliberately visited phishing or security-testing material. Two stated that they had not intentionally visited phishing pages, while the remaining browsing context was not separately confirmed. This context matters because an alert

produced during deliberate phishing testing cannot automatically be called a false alarm.

The extension records a unique canonical URL assessment rather than every model execution. Repeated checks of the same canonical URL therefore do not increase the unique-URL counter. A unique URL is not the same as a unique website: different paths under one hostname can be counted separately. This distinction is especially important for the Edge participant, who mainly used five or six websites but generated 2,524 distinct URL paths.

Table 7.2 retains the participant-level results. "Explicit FP" means that the participant used the false-alarm feedback action. "Confirmed FP" is used only for the Brave participant, who stated that no phishing page was visited and confirmed all 36 alerts as false, although the individual URLs were not stored. The four participants marked with an asterisk deliberately tested phishing or security URLs.

Table 7.2: Participant-level browser user-evaluation results.

Participant	Context	Unique URLs	Alerts	Explicit FP	Confirmed FP	Reported FN
Brave(S)	No intentional phishing	218	36	0	36	0
Chrome(A)*	Phishing/security testing	221	45	19	0	0
Chrome(AL)	Context unconfirmed	250	45	15	0	0
Chrome(B)	Context unconfirmed	702	55	10	0	0
Chrome(Mo)*	Phishing/security testing	201	46	18	0	3
Chrome(Mo2)*	Phishing/security testing	301	54	20	0	4
Chrome2(M)	No intentional phishing	202	21	10	0	0
Chromium(M)	Context unconfirmed	276	30	10	0	0
Edge(AL)	Context unconfirmed	2,524	18	2	0	0
GX(A)*	Phishing/security testing	407	68	34	0	0
Total		5,302	418	138	36	7

Note: An asterisk (*) identifies a participant who intentionally tested phishing or security-related URLs.

7.6.2 Recorded outcomes

The current-format exports contained 5,302 unique URL observations and 418 caution, warning, or block decisions. Users submitted 138 explicit false-alarm reports and seven missed-phishing reports. The explicit user-reported false-alarm rate was therefore $138/5302 = 2.60\%$, while the overall alert rate was $418/5302 = 7.88\%$. Including the 36 aggregate false alerts confirmed by the Brave participant gives a descriptive lower-bound false-alarm burden of $174/5302 = 3.28\%$.

Two earlier version 0.5.0 exports were preserved rather than discarded. They contained 41 alerts, six explicit false-alarm reports, and no reported missed threats. Their schema did not provide a compatible unique-URL denominator, so they are excluded from the rates above. Across current and legacy exports, explicit reports contained 144 false alarms and seven missed threats. Adding the 36 confirmed aggregate Brave events gives 180 observed false-alarm events, but it does not make the unreported

Table 7.3: Summary of current-format browser user-evaluation observations.

Recorded quantity	Count or rate
User exports with a current-schema URL denominator	10
Unique URL observations	5,302
Caution, warning, or block decisions	418
Explicit false-alarm reports	138
Explicit missed-phishing reports	7
Explicit user-reported false-alarm rate	2.60%
Overall alert rate	7.88%
False-alarm lower bound including confirmed aggregate events	3.28%

alerts correct by assumption.

These values are not classifier accuracy, precision, recall, FPR, or FNR. Labels were supplied only when a user reported an outcome, and they were not independently adjudicated in every case. The observations are therefore used to identify warning burden, repeated false-positive patterns, and interface problems. Formal model metrics remain based on the labelled frozen datasets in Chapter 6.

7.6.3 Reported error categories

The feedback nevertheless revealed useful error groups. Frequently reported cases included login and account pages, technical documentation, long service URLs, Google Workspace links, AI services, and security-learning websites. These findings led to exact-host display in the allow action, simpler notices, deduplicated counting, the reviewed-service catalogue, and the corrected shared-content policy. User feedback was therefore useful for improving the browser layer even though it could not replace labelled classifier evaluation.

Table 7.4: Categories of retained user-reported or participant-confirmed outcomes.

Category	False alarms	Missed threats
Unspecified (individual URL not stored)	36	0
Login, account, and webmail	25	0
Other websites	24	0
Technical documentation and research	21	0
Work and online services	17	0
Google and Workspace	15	0
AI services	13	0
Security learning and testing	13	0
Shopping, media, and games	11	0
Education, government, and jobs	5	0
Reported missed threat	0	7
Total	180	7

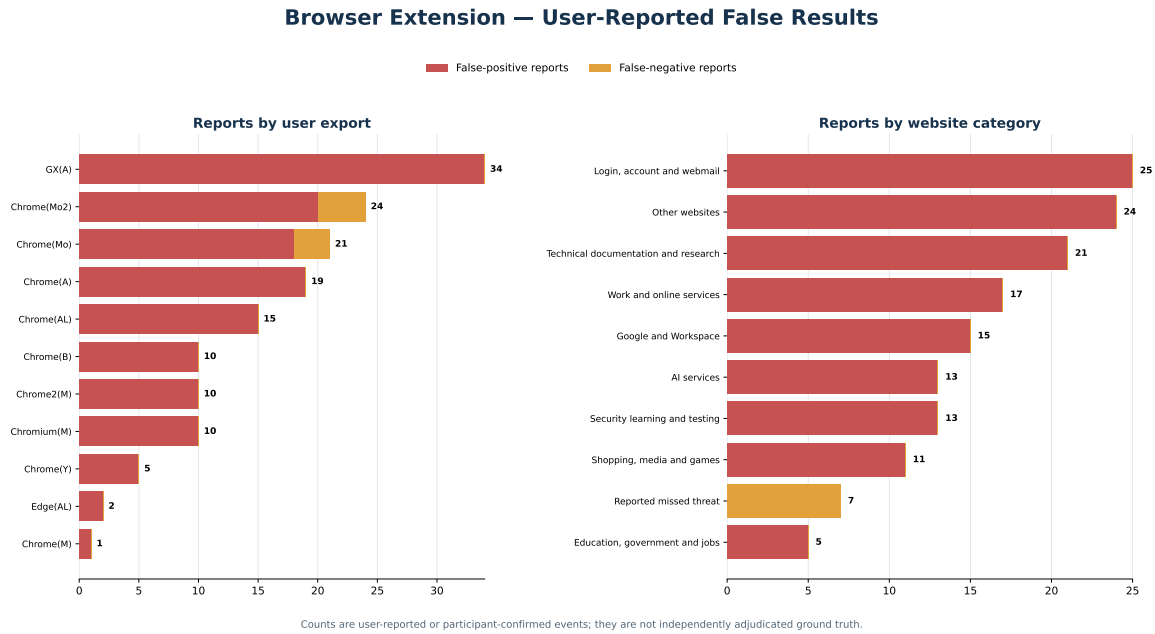


Figure 7.4: Retained user-reported and participant-confirmed outcomes by export and website category. These counts are feedback observations, not independently verified test labels.

7.7 Technical Verification

The extension build passed preprocessing parity tests, ONNX numerical parity tests, policy and localisation tests, privacy and deduplication tests, JavaScript syntax checks, a headless Chrome smoke test, and browser-navigation tests using locally fulfilled pages. ONNX score disagreement with the Python implementation was below 2.4×10^{-7} in the registered parity test. The current policy test confirms fixed-service handling, non-blocking caution notices, blocking of high-risk tiers in Normal mode, English and Arabic operation, and exact-host allow behaviour.

Passing these tests establishes implementation consistency, not permanent deployment readiness. The extension remains a graduation-project prototype whose URL-only model can produce false warnings and miss phishing under later distributions. Reputation, page content, certificates, and live infrastructure are outside the deployed classifier's evidence.

7.8 Chapter Summary

The browser extension runs the selected three-seed hybrid ensemble locally and turns its score into tiered, user-controllable actions. Reliability controls prevent repeated browser events from inflating the recorded number of unique URLs, while consent and local storage limit collection of browsing data.

Normal ChatGPT conversation routes are recognised service pages, while public

shares and custom GPT routes remain checked. User feedback identified recurring false-warning categories and guided usability improvements, but labelled frozen sets remain the basis for formal performance claims.

Chapter 8

Conclusion

8.1 Project Overview

This project investigated whether GAN-generated phishing URLs can improve a URL-based phishing detector and whether the selected detector can be used in a working browser extension. The work began with real-only classifiers, continued with several URL generators and controlled augmentation experiments, and ended with external evaluation and browser implementation.

The final classification approach, HYB-MS, combines a character-level CNN with 35 lexical URL features. The character branch learns local patterns in the URL text, while the lexical branch represents measurements such as length, digit and symbol ratios, subdomain structure, path depth, and query structure. The model was trained as a three-seed ensemble on the registered multi-source real training data. A matching model was then trained with an additional 10% filtered MS-HardGAN-C phishing URLs. Keeping the architecture, real rows, evaluation data, and calibration procedure fixed allowed the effect of GAN augmentation to be measured fairly.

The selected model was exported to ONNX and integrated into a Chromium browser extension. The extension performs local inference, presents several warning levels, supports English and Arabic, and allows the user to continue or permit an exact hostname. Reliability controls reduce repeated counting, and user-evaluation data remain local unless a participant chooses to export them.

8.2 Answers to the Research Questions

8.2.1 RQ1: Performance of real-only classifiers

The real-only experiments showed that URL phishing can be detected effectively when the training and test distributions are similar. The CNN and LSTM reached internal F1-scores above 97%. However, the external evaluations separated the models more

clearly. Some models retained a very low FPR by classifying few URLs as phishing, but consequently missed most phishing samples. This showed that accuracy or FPR alone is not sufficient for model selection.

Under equal calibration, LSTM-MS was the strongest ordinary neural baseline. HYB-MS real-only achieved a better overall external balance because it combined learned character patterns with full-URL lexical structure. On the primary 30K low-prevalence evaluation, real-only HYB-MS achieved 79.06% F1 and 81.07% recall, compared with 74.93% F1 and 75.20% recall for LSTM-MS.

8.2.2 RQ2: Quality of generated phishing URLs

The GAN experiments showed that uniqueness alone does not establish quality. A generator can produce many different strings that are invalid, structurally abnormal, repetitive in pattern, or easy to distinguish from real phishing URLs. Generated pools were therefore evaluated using validity, uniqueness, real-data overlap, lexical realism, diversity, source discrimination, and their ability to challenge frozen classifiers.

SeqGAN was the strongest of the initial text generators, while MS-HardGAN-C provided the final filtered augmentation pool. The accepted pool contained no exact overlap with the permitted real training URLs. The generator did not predict future zero-day attacks; instead, it recombined patterns learned from the available phishing training data to provide additional difficult examples.

A later matched-input experiment retrained SeqGAN on the exact same 84,280 phishing rows used by MS-HardGAN-C. This improved SeqGAN's downstream HYB-MS result, showing that generator input data mattered. MS-HardGAN-C still achieved the stronger recall, F1, and FNR balance on the frozen 10K and 30K comparisons. Its selection is therefore supported by a direct input-controlled comparison, not only by the earlier studies in which the generators used different phishing pools.

8.2.3 RQ3: Effect of GAN augmentation

GAN augmentation did not improve every model, ratio, or metric. Random augmentation and higher synthetic percentages frequently reduced performance on the unchanged real test data. These results are important because they show that simply increasing the quantity of generated URLs is not a reliable method of improving detection.

The clearest benefit came from the controlled 10% MS-HardGAN-C condition. On the primary 30K external dataset, HYB-MS+10% achieved 98.12% accuracy, 80.95% precision, 81.60% recall, 81.27% F1, 96.82% ROC-AUC, and 87.27% PR-AUC. It also reduced FPR from 1.26% to 1.01% and FNR from 18.93% to 18.40% relative to real-only HYB-MS. The paired F1 improvement was supported by the reported

confidence interval and McNemar comparison, although the recall change alone was not statistically significant.

The result answers the research question positively but conditionally: carefully filtered and limited GAN augmentation can improve the balance of URL phishing detection under the evaluated conditions. The experiments do not show that augmentation always improves performance.

The matched-generator experiment also showed why the generator choice matters. On the balanced external 10K set, matched SeqGAN reached 91.59% F1 and 14.36% FNR, while MS-HardGAN-C reached 92.67% F1 and 12.48% FNR at the same 1.36% FPR. Both generators had received the same real phishing training identities and both supplied exactly 40,410 synthetic rows to HYB-MS.

8.2.4 RQ4: Generalisation to different and imbalanced sources

The selected 10% model improved F1 and reduced both FPR and FNR relative to its real-only control on several frozen low-prevalence external datasets. These included the PhishTank 10K set, the final labelled 6K set, the post-freeze 10K set, and the primary 30K set. The final labelled 6K evaluation contained only 5% phishing and produced 98.23% accuracy, 84.15% precision, 79.67% recall, 81.85% F1, 0.79% FPR, and 20.33% FNR for HYB-MS+10%.

The external scores were lower than the strongest internal scores. This is evidence of distribution shift rather than a contradiction: later URLs came from different sources and contained structures that were less represented in training. The long-URL test also showed a trade-off, where augmentation found additional phishing URLs but introduced additional false warnings. Therefore, the model generalises usefully across the tested sets, but it does not remove distribution shift.

8.2.5 RQ5: Browser implementation

The selected ensemble was successfully deployed as a local browser-extension prototype. Preprocessing parity, ONNX numerical parity, navigation handling, localisation, consent, deduplication, policy, and browser smoke tests were completed. The extension separates the raw model score from the action shown to the user, allowing moderate evidence to produce a non-blocking notice while reserving navigation interruption for the strictest tier.

The browser study also showed why extension policy must be evaluated separately from classifier quality. Blanket trust for Google Workspace suppressed warnings for phishing hosted in user-controlled documents. Replacing it with a capped, non-blocking policy recovered 24 detections on the controlled snapshot and reduced FNR from 21.33% to 13.33% without changing the measured FPR. User feedback further identified

false warnings on login pages, long service URLs, technical sites, Google services, and AI services. The extension satisfies the project implementation objective as a research prototype, but it is not presented as a certified production security product.

8.3 Achievement of the Project Objectives

Table 8.1: Final status of the objectives defined in Chapter 1.

Objective	Evidence completed in the project	Status
O1	Label cleaning, canonicalisation, class counts, length analysis, duplicate removal, and identity-disjoint splits.	Achieved
O2	RF, SVM, CNN, RNN, LSTM, and hybrid real-only classifiers trained and evaluated.	Achieved
O3	URL GAN variants trained only with the permitted phishing training data.	Achieved
O4	Validity, uniqueness, overlap, realism, diversity, source, and challenge analyses completed.	Achieved
O5	Real-only and fixed-ratio augmented conditions compared with controlled rows, seeds, and test sets.	Achieved
O6	Internal, source-held-out, later, low-prevalence, PhishTank, long-URL, and GAN-challenge tests completed.	Achieved
O7	Selected ensemble exported to ONNX and implemented in a locally evaluated browser extension.	Achieved

All seven objectives were completed within the research scope. Achievement of the deployment objective means that a functioning browser prototype was built and evaluated. It does not mean that the system has completed the security, privacy, device-compatibility, and maintenance work required for unrestricted public deployment.

8.4 Main Findings

The main findings of the project are as follows:

- Internal random-split performance can overestimate performance on URLs from later or different sources.
- Combining character learning and lexical structure gave the hybrid a better external balance than the ordinary baselines under equal calibration.
- Generated URL quality must be evaluated separately from downstream classifier performance. High uniqueness does not guarantee realistic or useful augmentation.

- Large or unfiltered synthetic additions were often harmful. The best evidence supported a limited, filtered 10% augmentation condition.
- The benefit of the selected GAN augmentation was repeated on several low-prevalence frozen datasets, but its size varied by dataset.
- Thresholds control the FPR–FNR trade-off. Lower FNR can be obtained by accepting more false warnings, but no threshold removes distribution overlap.
- Browser policy can either expose or hide model detections. Trusted platforms must not silently override strong model evidence on user-controlled content.
- User feedback is valuable for finding usability problems, but formal accuracy metrics require labelled evaluation data rather than incomplete self-reported outcomes.

8.5 Limitations

The detector uses URL text and lexical structure only. It does not inspect the page DOM, visual design, certificate chain, DNS history, domain age, network behaviour, or reputation feeds. A convincing phishing URL can therefore be missed, while a long or unusual legitimate URL can receive a warning.

The evaluation datasets cannot represent every future campaign, language, country, hosting platform, or browsing pattern. Labels from public feeds can also contain delays or errors. Although repeated seeds and frozen comparisons improve reproducibility, the GAN and classifier findings remain specific to the implemented models and available data.

The browser user evaluation was useful for interface and false-warning analysis, but it was not a fully supervised clinical-style study. Not every visited URL received an independently verified label, devices and browsing patterns differed, and some participants deliberately tested phishing. Its counts must therefore remain separate from the formal model metrics.

8.6 Final Conclusion

The project provides a valid experimental answer to its main question. GAN augmentation can improve phishing URL detection when generated samples are filtered, limited, and compared fairly with a matching real-only model. The 10% MS-HardGAN-C augmented hybrid produced the strongest and most consistent external balance in this study. At the same time, the negative augmentation results and remaining FNR show that synthetic data is not a replacement for diverse real data.

The completed browser extension demonstrates that the research model can run locally and support real-time URL warnings. Its tiered decisions, reviewed service rules, and user controls make it suitable for graduation-project demonstration and further supervised testing. Continued data collection and multi-signal verification would be required before treating it as a production-ready phishing defence.

Chapter 9

Future Work

9.1 Introduction

The project completed its proposed experimental and implementation objectives, but phishing methods and normal web services continue to change. Future work should therefore concentrate on maintaining the detector, reducing avoidable warnings, and evaluating the browser extension with broader evidence. The following improvements extend the current system without changing the main conclusion of this report.

9.2 Newer and Broader URL Data

The highest priority is to collect labelled URLs at different times after the current dataset freeze. Later snapshots should contain verified phishing URLs, ordinary browsing, long legitimate URLs, login and account pages, Arabic and Libyan services, cloud documents, and user-controlled content hosted on trusted platforms. Collection time and source should be stored so that later data can be used as a true external test rather than mixed into the existing test sets.

When sufficient new data have been reviewed, a later model may be retrained using a new training split. The present frozen tests should still be retained for historical comparison, while an additional untouched snapshot becomes the new final test. This would show whether an improvement transfers across time instead of only fitting one evaluation period.

9.3 Reducing False Warnings and Missed Phishing

Future experiments can examine the errors with the highest practical cost. Legitimate examples should include difficult login, redirect, documentation, search, cloud-storage, and long-query URLs. Difficult phishing examples should include compromised services,

shared documents, short paths, and URLs whose structure resembles a normal service.

Small controlled experiments with class weighting, focal loss, and hard-example mining could test whether the score distributions become better separated. Thresholds should continue to be chosen on validation data under stated FPR constraints. The final test must remain untouched until the choice is frozen. This preserves a fair comparison and avoids lowering FNR by simply accepting an unusable number of false warnings.

9.4 Additional Detection Evidence

URL-only analysis is fast and privacy-friendly, but it cannot observe every property of a website. A later system could combine the current URL model with lightweight independent signals, including:

- domain and DNS history;
- certificate and HTTPS information;
- domain reputation and verified phishing feeds;
- safe HTML or DOM features collected after explicit permission; and
- redirect-chain and final-destination information.

These signals should be added as a secondary review layer rather than presented as replacements for the GAN-augmentation experiment. A controlled comparison of real-only and GAN-augmented primary models with the same secondary evidence would show whether the augmentation benefit remains present.

9.5 Further GAN Research

Future generator work could use newer verified phishing data and compare several generator checkpoints over time. Evaluation should retain the existing validity, uniqueness, overlap, diversity, realism, and classifier-challenge checks. Human review of a small defanged sample could be added with a clear review protocol.

The selection rule should be fixed before classifier testing. Generated URLs should not be selected because they happen to improve a final test result. Experiments may also compare whether augmentation is more valuable when less real phishing training data are available. Such work would clarify when synthetic data supplies useful coverage and when additional real data is the better choice.

9.6 Browser Extension Improvements

The browser prototype can be improved through:

- clearer explanations of caution, warning, and block decisions;
- broader testing of Arabic localisation and right-to-left layouts;
- accessibility checks for colour, keyboard use, and screen readers;
- lower memory and initialization cost on older computers;
- clearer management and expiry of user-allowed hostnames;
- versioned and auditable rules for fixed services and user-controlled content; and
- safe model and policy updates with rollback support.

Before browser-store publication, the extension would require a dedicated security review, permission review, privacy documentation, dependency audit, and testing on a wider range of Chromium versions and devices. Public release should also provide a clear statement that the extension is an additional warning tool and cannot guarantee that a page is safe.

9.7 Larger User Evaluation

A later user study should recruit more participants and use a written test procedure. Participants could complete a small set of labelled tasks followed by a period of ordinary browsing. This would permit separate measurement of task accuracy, warning comprehension, false-warning burden, time to decision, and user satisfaction.

Consent should remain explicit, and raw browsing data should remain local by default. If labelled feedback is submitted, the user should see exactly what will be shared. Repeated paths from the same website should be reported separately from unique hostnames so that one service does not dominate the study counts.

9.8 Long-Term Maintenance

Phishing detection is not a one-time training task. A future operational version would need periodic data-quality checks, source-drift monitoring, threshold review, generator evaluation, and model rollback. Metrics should be reported by source and time period as well as in aggregate. Sudden changes in FPR, FNR, or the score distribution should trigger investigation before a new model is released.

9.9 Chapter Summary

The most useful next work is to evaluate later labelled URL snapshots, improve coverage of difficult legitimate and phishing cases, and conduct a larger supervised browser study. Adding DNS, certificate, reputation, redirect, or page evidence may further reduce the limitations of URL-only detection. However, every change should preserve the controlled comparison used in this project: fixed training conditions, validation-based decisions, untouched test sets, and transparent reporting of both improvements and failures.

References

[1] Ozgur Koray Sahingoz, Ebubekir Buber, Onder Demir, and Banu Diri. Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117:345–357, 2019.

[2] Sharif Amit Kamran, Shamik Sengupta, and Alireza Tavakkoli. Semi-supervised conditional GAN for simultaneous generation and detection of phishing URLs: A game theoretic perspective. *arXiv preprint arXiv:2108.01852*, 2022.

[3] Trinh Nguyen Bac, Phan The Duy, and Van-Hau Pham. PWDGAN: Generating adversarial malicious URL examples for deceiving black-box phishing website detector using GANs. In *Proceedings of the International Conference on Machine Learning and Applied Network Technologies*, 2021.

[4] Thi Thanh Thuy Pham, Tuan Dung Pham, and Viet Cuong Ta. Evaluation of GAN-based models for phishing URL classifiers. *International Journal of Computer Network and Information Security*, 15(2):1–14, 2023.

[5] Saad Al-Ahmadi, Afrah Alotaibi, and Omar Alsaleh. PDGAN: Phishing detection with generative adversarial networks. *IEEE Access*, 10:42459–42470, 2022.

[6] Abdullah Al-Alyan and Saad Al-Ahmadi. Robust URL phishing detection based on deep learning. *KSII Transactions on Internet and Information Systems*, 14(7):2752–2768, 2020.

[7] Min-Yen Kan and Hoang Oanh Nguyen Thi. Fast webpage classification using URL features. In *Proceedings of the 14th ACM International Conference on Information and Knowledge Management*, 2005.

[8] Yichen Wang. Malicious URL detection: An evaluation of feature extraction and machine learning algorithm. *Highlights in Science, Engineering and Technology*, 23:117–123, 2022.

[9] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In

Advances in Neural Information Processing Systems, volume 27, pages 2672–2680, 2014.

[10] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *International Conference on Learning Representations*, 2017.

[11] Lantao Yu, Weinan Zhang, Jun Wang, and Yong Yu. SeqGAN: Sequence generative adversarial nets with policy gradient. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

[12] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein GANs. In *Advances in Neural Information Processing Systems*, volume 30, pages 5767–5777, 2017.

[13] Marwa Abd Al Hussein Qasim and Nahla Abbas Flayh. Phishing website detection using machine learning: A review. *Wasit Journal for Pure Sciences*, 2(2), 2023.

[14] Tuan Dung Pham, Thi Thanh Thuy Pham, Sy Tuong Hoang, and Viet Cuong Ta. Exploring efficiency of GAN-based generated URLs for phishing URL detection, 2021. Research paper supplied with the project literature.

[15] Putra Wanda, Mohammad Diqi, Hamzah, Irma Permata Sari, and Selamet Riadi. Deepphishing: Stepping up phishing detection using generative round network. *International Journal of Information Technology*, 2025.

[16] PhishTank. Developer information and phishing data feed. `https://phishtank.org/developer_info.php`, 2026. Verified phishing URL data source.